

INDIAN INSTITUTE OF TECHNOLOGY KANPUR

DEPARTMENT OF MECHANICAL ENGINEERING

SMART MATERIALS, STRUCTURES AND SYSTEMS (SMSS) LABORATORY

SURGE 2026 INTERNSHIP REPORT

Vision-Guided UAV Precision Landing on Static and Mobile UGV Platforms Using Predictive and Optimal Control Strategies

Submitted by:

Lavanya Bhatnagar

SURGE Roll No. S260081

Under the guidance of:

Prof. Bishakh Bhattacharya

Dept. of Mechanical Engineering

IIT Kanpur

SURGE 2026 (11 May – 10 July 2026)

Abstract

Precision autonomous landing remains a critical challenge in UAV deployment, demanding accurate perception, robust state estimation, and responsive control under real-world disturbances. Landing a UAV on a moving ground platform introduces additional complexity through dynamic relative motion between agents, the need for real-time pose estimation under onboard camera constraints, and accurate trajectory tracking of a non-stationary target. These combined requirements expose the fundamental limitations of conventional reactive control approaches, motivating the adoption of model-based strategies that can anticipate platform motion and enforce kinematic constraints during descent.

This work presents the design, simulation, and hardware implementation of a vision-guided UAV precision landing system using ArUco marker-based 6-DoF relative pose estimation, developed using the PX4 autopilot stack with a Pixhawk flight controller, ROS2 middleware, and the Gazebo simulation environment. A downward-facing camera mounted on the UAV estimates the relative pose of the moving platform in real time, feeding into high-level controllers that generate corrective velocity commands to guide the drone through a stable descent trajectory toward the marker. To investigate the suitability of different control strategies for moving platform landing, a comparative study of PID, Linear Quadratic Regulator and Model Predictive Control is undertaken. All three controllers are first developed and validated in simulation against a stationary ArUco marker and subsequently extended to a moving platform, before hardware deployment on a Holybro S500 quadrotor with NVIDIA Jetson Orin Nano as the companion computer. Hardware commissioning includes sensor calibration, RC link verification, GPS communication testing, and evaluation of offboard control mode. Controller performance is assessed using metrics including landing accuracy, tracking error, convergence time, control effort smoothness, and robustness to platform motion. The findings of this study are envisaged to contribute toward the broader development of reliable autonomous precision landing systems, with applications spanning aerial logistics, mobile platform operations, and any scenario demanding accurate UAV touchdown on dynamically moving targets, providing a systematic simulation-to-hardware evaluation of model-based control strategies for vision-guided autonomous UAV landing.

Acknowledgements

The author sincerely expresses her heartfelt gratitude to the Smart Materials, Structures and Systems (SMSS) Laboratory, IIT Kanpur, for providing an excellent research environment, access to laboratory resources, hardware platforms, experimental facilities, and technical infrastructure that made this work possible. The author is deeply grateful to Prof. Bishakh Bhattacharya for his invaluable guidance, encouragement, and insightful suggestions throughout the course of this internship. Special thanks are extended to Mr. Shashank Mani Gautam for his constant mentorship, patience, and unwavering support. His technical guidance, thoughtful discussions, and willingness to share his expertise were instrumental in overcoming numerous practical and research challenges encountered during this project. His encouragement and continuous feedback significantly contributed to both the successful completion of this work and the author's learning experience.

The author also gratefully acknowledges everyone at the SMSS Laboratory whose support and cooperation contributed, directly or indirectly, to the successful completion of this internship.

Contents

Abstract	i
Acknowledgements	ii
Nomenclature	viii
1 Introduction	1
1.1 Motivation and Project Context	1
1.2 Problem Statement	1
1.3 Objectives and Scope	1
1.4 Contributions	2
2 Background and Related Work	3
2.1 Vision-Guided Landing Approaches	3
2.1.1 The perception problem	3
2.1.2 Fiducial markers	3
2.1.3 Image-based versus position-based visual servoing	3
2.1.4 Perception under motion and in the field	4
2.2 Control Strategies for UAV Landing	4
2.2.1 PID and reactive control	4
2.2.2 LQR and LQG: optimal linear control	4
2.2.3 Model Predictive Control	4
2.2.4 Nonlinear MPC and beyond	5
2.2.5 Robust and sliding-mode control	5
2.2.6 Learning-based control	5
2.2.7 Reviews	5
2.3 Positioning of This Work	5
2.3.1 Rationale for the selected controllers	5
2.3.2 The shared linear model and comparability	6
2.3.3 Scope of the excluded methods	6
2.3.4 Relation to the moving-platform literature	6
3 System Architecture and Simulation Environment	7
3.1 Hardware Platform Overview	7
3.2 Software Stack	7
3.3 Vision Pipeline	8
3.4 Communication and Offboard Control Interface	9
3.5 Simulation Scenarios	11
4 UAV Dynamics and Plant Modeling	15
4.1 Coordinate Frames and Conventions	15
4.2 Nonlinear Rigid-Body Dynamics	15
4.3 Hover Linearization and Axis Decoupling	15
4.4 Inner-Loop Abstraction: First-Order Velocity Lag	16
4.5 Time-Constant Identification	17
4.6 Error State-Space Formulation	17
4.7 Controllability Analysis	18

4.8	Modelling Assumptions and Neglected Effects	18
5	State Estimation	19
5.1	Motivation for Filtering	19
5.2	Kalman Filter Design	19
5.3	Process and Measurement Noise Selection	20
5.4	Consistency Validation via NIS	20
5.5	The Separation Principle and LQG	21
5.6	Role of the Estimator in the Comparative Study	21
6	Controller Design	22
6.1	Comparison Methodology	22
6.2	PID Control	22
6.3	LQR and LQG	22
6.4	Model Predictive Control	24
6.5	Constraint Handling: Clipping versus Hard Constraints	24
7	Results and Comparative Analysis: Static Marker	25
7.1	Metrics and Evaluation Protocol	25
7.2	PID Results (Reactive Baseline)	25
7.3	LQR Results	26
7.4	MPC Results	26
7.5	LQR versus MPC: Comparative Analysis	28
8	Extension to Moving-Platform Landing	33
8.1	Scenario and Reference-Tracking Formulation	33
8.2	LQR Steady-State Lag Against a Ramp	33
8.3	MPC Anticipation via the Moving Reference	33
8.4	Scope of Results and Discussion	34
9	Hardware Commissioning	35
9.1	Sensor, Motor, and Radio Calibration	35
9.2	Vision Pipeline: Exposure Robustness	35
9.3	State Estimation: GPS and Barometer	35
9.4	Companion-Computer Integration and Communication	36
9.5	Offboard-Mode Commissioning	36
9.6	Status and Path to Closed-Loop Trials	37
10	Discussion	39
10.1	Controller Trade-offs	39
10.2	Limitations and Confounds	39
10.3	Assumptions Revisited	39
11	Conclusions and Future Work	41
11.1	Conclusions	41
11.2	Future Work	41

List of Figures

1.1	The integrated UAV–UGV system developed for autonomous power-grid inspection. A Holybro S500 quadrotor operates from an unmanned ground vehicle that serves as a mobile landing, transport, and recharging platform.	2
3.1	Power distribution architecture of the UAV hardware platform.	8
3.2	Vision pipeline for ArUco-based relative pose estimation and coordinate transformations.	9
3.3	Hardware communication architecture showing sensor, telemetry and companion computer interfaces.	10
3.4	Sequence of data exchange during PX4 Offboard operation through the Micro XRCE-DDS bridge.	11
3.5	Internal cascaded flight control architecture implemented within PX4.	12
3.6	Software-in-the-Loop (SITL) architecture integrating Gazebo, ROS 2, PX4 Autopilot and the Micro XRCE-DDS communication bridge.	13
3.7	Representative Gazebo simulation scenarios used for controller development and validation.	14
4.1	Coordinate frames. The body frame $\mathcal{B}$ is fixed to the vehicle with its origin at the centre of mass ($O_B \equiv P$); the rotation $R_{\mathcal{B} \rightarrow \mathcal{I}}$ maps body-frame quantities into the inertial frame $\mathcal{I}$ under the NED convention.	16
5.1	Kalman-filter velocity estimate ($\hat{v}_x, \hat{v}_y$) against the true PX4 velocity. Velocity is not measured; it is inferred through model propagation, and the estimate follows the truth closely.	20
5.2	NIS consistency check with the measured R . The shaded region is the 95% χ^2_2 band $[0.05, 7.38]$; the dotted line marks the expected mean of 2. The measured mean lies well below 2, indicating a conservative filter.	21
6.1	PX4 inner proportional–integral velocity loop, abstracted in Chapter 4 as a first-order lag with time constant τ	23
6.2	Discrete LQR guidance loop. The regulator gain $-K_d$ acts on the error state to produce the velocity setpoint passed to the inner loop.	23
7.1	PID design study: horizontal error versus time for the proportional (P), proportional–derivative (PD), and full PID controllers. State source: Gazebo GT.	26
7.2	Detailed settling-phase comparison ($t > 20$ s) for the P, PD, and PID controllers. Derivative and integral action progressively suppress the residual oscillation. State source: Gazebo GT.	27
7.3	PID tuning-sweep metrics across proportional, derivative, and integral gain settings. Highlighted cells mark the best settling time, 2-D RMSE, and control effort in each group.	28
7.4	LQR control-effort weighting study: static-marker error response for a sweep of R , with $R = 5$ the adopted baseline. State source: Gazebo GT.	29
7.5	MPC accuracy versus prediction horizon N : RMSE, steady-state error, and final touchdown error, for $Q_f = Q$ (no terminal cost) and $Q_f = P$ (DARE terminal cost). Error metrics use the KF-estimated state.	30

7.6	Predicted versus realized error paths for $N \in \{10, 20, 40\}$: DARE terminal cost $Q_f = P$ (left) and no terminal cost $Q_f = Q$ (right). Dashed lines are the per-snapshot planned horizons; solid lines are the realized path. State source: KF. . .	31
7.7	Realized 3-D descent paths for $N \in \{10, 20, 40\}$: $Q_f = P$ (left) and $Q_f = Q$ (right). Longer horizons and the DARE terminal cost yield straighter descents. State source: KF.	31
7.8	Kalman-estimate versus PX4-actual velocity for LQR and MPC under identical R and Q : (a) LQR $R = 5$; (b) MPC $N = 20$, $Q_f = P$; (c) MPC $N = 20$, $Q_f = Q$. The infinite-horizon MPC tracks most closely; LQR shows the largest mismatch.	32
7.9	Control effort $\int u^2 dt$ versus prediction horizon N for MPC with $Q_f = Q$ and $Q_f = P$, with the LQR ($R = 5$) effort shown as a single reference point.	32
9.1	Vehicle bring-up in QGroundControl. Sensor, motor, and radio calibration established a stable armed baseline before autonomy testing.	36
9.2	Flight-mode and switch assignment, including a dedicated Offboard channel and an emergency-kill switch for safe manual override.	36
9.3	Effect of exposure tuning on marker detection. Left: under auto-exposure the frame overexposes and no marker is detected. Right: with manual exposure tuning the marker is reliably detected and its relative pose recovered.	37
9.4	Micro XRCE-DDS communication pipeline. The left window shows the PX4 shell confirming the Micro XRCE-DDS client connection with the matching <code>ROS_DOMAIN_ID</code> ; the middle window shows the running Micro XRCE-DDS Agent; and the right window displays the corresponding PX4 uORB topics successfully bridged and communicated as ROS 2 topics.	38

List of Tables

7.1	LQR performance metrics for the control-effort weighting sweep, with $R = 5$ the adopted baseline. State source: Gazebo ground truth.	26
7.2	Infinite-horizon-equivalent MPC ($Q_f = P$, $R = 5$): performance metrics versus prediction horizon N . Error metrics use the KF-estimated state. “—” denotes the 0.1 m settling band not reached within the run.	28
7.3	Short-horizon MPC ($Q_f = Q$): the same metrics versus prediction horizon N . Error metrics use the KF-estimated state. “—” denotes the 0.1 m settling band not reached within the run.	29
9.1	Tuned camera controls (baked into the camera launch file) versus factory defaults, with the rationale for each change.	37

Nomenclature

$x_{\text{err}}, y_{\text{err}}$	Horizontal position tracking error (marker — drone)
τ	First-order velocity-loop time constant
$\mathbf{A}, \mathbf{B}$	Continuous state and input matrices
$\mathbf{Q}, \mathbf{R}$	LQR state and input weighting matrices
$\mathbf{K}$	LQR feedback gain
N	MPC prediction horizon
NIS	Normalized Innovation Squared
PBVS	Position-Based Visual Servoing

1 Introduction

1.1 Motivation and Project Context

This work forms part of a broader effort on autonomous inspection of power-transmission infrastructure. Power lines, towers, and substations require frequent inspection for faults, corrosion, and structural degradation. UAVs improve inspection efficiency but remain limited by flight endurance. but sustained inspection over large areas is fundamentally bounded by a UAV’s limited flight endurance. A recurring architecture that addresses this limitation is a UAV–UGV team: an unmanned ground vehicle (UGV) carries, transports, and recharges the UAV, while the UAV performs the aerial inspection, extending effective mission range far beyond what either agent could achieve alone.

The enabling capability for such a team is *autonomous precision landing*: the UAV must reliably return to and land on the ground vehicle, whether that vehicle is stationary or in motion. This report addresses that landing problem as a self-contained sub-problem of the broader power-grid inspection project. Precise, repeatable landing on the platform — ultimately a *moving* platform — is the linchpin that makes autonomous recharging and redeployment possible. The physical UAV–UGV system developed for this project is shown in Fig. 1.1.

1.2 Problem Statement

The core problem is to guide a UAV, using only onboard sensing, to a precise landing on a target platform marked with a visual fiducial. A downward-facing camera estimates the UAV’s position relative to an ArUco marker mounted on the platform; a high-level controller must drive this relative position to zero while managing a stable descent and respecting the vehicle’s kinematic limits.

Two variants define a natural progression in difficulty. In the *static* case the platform is stationary and the landing reference is fixed — a regulation problem. In the *moving* case the platform translates, and the controller must track a time-varying reference, anticipating platform motion rather than merely reacting to it — a tracking problem. The moving case exposes the limits of purely reactive control and motivates model-based strategies that can incorporate the platform’s motion. This report establishes and rigorously evaluates the static-platform foundation and the control framework required for both cases, and formulates the moving-platform extension as the immediate next stage of the work.

1.3 Objectives and Scope

The objectives of this work are:

1. To derive a control-oriented dynamic model of the UAV’s horizontal landing behaviour from first principles, with clearly stated linearization and decoupling assumptions.
2. To design and implement three control strategies — PID, LQR/LQG, and MPC — for vision-guided precision landing.
3. To compare these controllers under a fair-comparison protocol in which all consume identical state information, so that measured differences are attributable to the control law rather than to the estimator or the inputs.
4. To validate the state estimator (a Kalman filter) as a prerequisite for LQG, using statistical consistency checks.



(a) UAV in flight above the UGV platform.



(b) UAV landed on the UGV platform.

Figure 1.1: The integrated UAV–UGV system developed for autonomous power-grid inspection. A Holybro S500 quadrotor operates from an unmanned ground vehicle that serves as a mobile landing, transport, and recharging platform.

5. To document hardware commissioning of the physical platform and the integration findings that inform eventual deployment.

The scope of this report is deliberately bounded. The comparative controller study and all quantitative results are established for the **static-platform** scenario in high-fidelity simulation (PX4 Software-in-the-Loop with Gazebo). The **moving-platform** scenario is treated at the level of problem formulation and the analytical control argument that distinguishes the candidate controllers; empirical moving-platform results are identified as the immediate next step and are not claimed here. Hardware work is reported as a commissioning effort rather than as closed-loop autonomous flight. Drawing these boundaries explicitly is itself part of the contribution: it separates what has been demonstrated from what has been designed for.

1.4 Contributions

This report makes the following contributions:

- A first-principles, control-oriented state-space model of the UAV landing dynamics, including identification of the inner-loop velocity time constant and an explicit controllability analysis.
- A *comparison methodology* for evaluating PID, LQR/LQG, and MPC on identical state information, removing the estimator asymmetry that confounds many controller comparisons.
- A validated Kalman-filter estimator, with noise parameters selected from measured sensor data and consistency confirmed via a Normalized Innovation Squared (NIS) test.
- A comparative analysis of the three controllers on the static-platform task, including MPC prediction-horizon and terminal-cost studies with a real-time-feasibility assessment.
- A documented account of hardware commissioning, including a reproducible resolution of an outdoor vision-exposure failure and the diagnosis of state-estimation and offboard-mode integration issues.

2 Background and Related Work

Autonomous precision landing sits at the intersection of two research threads: *perception* — estimating where the landing target is relative to the vehicle — and *control* — driving the vehicle onto that target under its physical limits. This chapter reviews both. Section 2.1 surveys vision-guided landing approaches and situates the position-based visual servoing used here. Section 2.2 surveys the control strategies applied to UAV landing, from reactive to predictive and robust methods. Section 2.3 then states, precisely and honestly, why this work selects the particular trio of controllers it compares, and what it deliberately leaves out.

2.1 Vision-Guided Landing Approaches

2.1.1 The perception problem

Precision landing requires knowing the pose of the landing target relative to the UAV, continuously and at a rate fast enough to close a control loop. The dominant sensing modalities differ sharply in where they work. Satellite positioning (GPS, or RTK-GPS for centimetre accuracy) is the default for outdoor waypoint navigation, but it degrades or fails precisely where inspection robots often operate — near large metal structures, under canopy, or in GPS-denied environments — and its accuracy is generally insufficient for landing on a small, possibly moving platform. Motion-capture systems provide millimetre-accurate pose and are common in laboratory demonstrations of moving-platform landing [12], but they are confined to instrumented spaces and cannot deploy in the field. These limitations motivate *onboard vision* as the sensing modality for practical precision landing: it is self-contained, works without external infrastructure, and provides the relative pose directly.

2.1.2 Fiducial markers

The most widely adopted vision approach places a known fiducial marker on the landing platform. Square-marker families such as ArUco and AprilTag are popular because they are cheap, robust to moderate viewpoint and lighting change, and — critically — allow full six-degree-of-freedom relative pose recovery from a single image once the marker’s physical size and the camera intrinsics are known. Detection returns the marker’s corner pixels; a perspective-n-point (PnP) solve then yields the marker pose in the camera frame. This makes fiducial-based landing a mature, reliable building block, and it is the approach adopted in this work: an ArUco marker on the platform, detected by a downward-facing camera, with metric relative position recovered via `solvePnP`. Open implementations of this pattern exist and informed the present pipeline [21].

2.1.3 Image-based versus position-based visual servoing

A key distinction in vision-guided control is *how* the visual measurement enters the controller. In image-based visual servoing (IBVS), the control error is defined directly in the image plane (in pixels), and the controller regulates image features to their goal positions; the 3-D geometry is never explicitly reconstructed. In position-based visual servoing (PBVS), the visual measurement is first converted into a metric 3-D pose (via PnP), and the control error is defined in Cartesian space (in metres). The two lead to different controller structures and stability properties. This work uses *PBVS*: because the ArUco pose is recovered in metres, the control error is a physical position error, which is what makes a linear, metric state-space plant model (Chapter 4) meaningful. This is an important modelling commitment — a claim of IBVS would be inconsistent with the metric, position-space plant used throughout.

2.1.4 Perception under motion and in the field

Two practical challenges recur. First, a moving platform makes the perception problem time-varying: the marker must be tracked, its velocity estimated or inferred, and measurement latency compensated, since even small delays translate into position error at speed. Second, outdoor operation stresses the camera itself — exposure, dynamic range, and frame rate all affect detection reliability, a point borne out in this project’s own hardware commissioning (Chapter 9), where uncontrolled exposure caused outright detection failure in direct sunlight. Learning-based detectors can relax the requirement for a cooperative marker, but at the cost of data, training, and a sim-to-real gap; for a known platform, fiducial markers remain the pragmatic choice.

2.2 Control Strategies for UAV Landing

The control literature for UAV landing spans a spectrum from purely reactive to fully predictive and robust. It is useful to read this spectrum as a progression in *how much model and foresight* the controller uses.

2.2.1 PID and reactive control

Proportional–Integral–Derivative control is the ubiquitous baseline. It is simple, model-free in the sense that it needs no explicit state-space model, and easy to tune, and it underlies the inner attitude and rate loops of essentially every autopilot including PX4. For landing, PID (or PD) laws on the relative position error are a common first approach and remain competitive for static targets [20]. Their limitations are structural, not incidental: a PID law cannot explicitly respect input or state constraints, has no notion of the future, and — for a moving target — tends to lag, because it reacts to the present error rather than anticipating the platform’s motion. These limits are exactly what motivate model-based control for the moving-platform case.

2.2.2 LQR and LQG: optimal linear control

The Linear Quadratic Regulator computes a state-feedback gain that minimizes a quadratic cost trading off tracking error against control effort, given a linear model of the plant. It is optimal (for that model and cost), yields a constant gain that is cheap to run, and provides guaranteed stability margins for the nominal linear system. LQR has been applied extensively to quadrotor control [9, 10] and to relative-position precision tasks [11]. When the full state is not directly measured, a state estimator — typically a Kalman filter — is combined with the LQR gain; the resulting output-feedback controller is Linear Quadratic Gaussian (LQG) control, and the separation principle guarantees that the optimal estimator and optimal regulator can be designed independently. LQR/LQG’s two structural limitations matter for landing: it cannot enforce hard constraints, and, being a fixed linear feedback, a standard LQR behaves as a type-1 loop that incurs a constant steady-state lag when tracking a constant-velocity (ramp) reference — the moving-platform case.

2.2.3 Model Predictive Control

Model Predictive Control repeatedly solves a finite-horizon optimal control problem online, applying the first control of the optimized sequence and re-planning each step. Its two defining advantages over LQR are precisely the two things LQR lacks: it can enforce hard constraints on states and inputs by construction, and it can incorporate a *preview* of the reference over the horizon — so a moving platform’s projected motion can be anticipated rather than chased. MPC for quadrotors and for landing has a substantial literature, spanning tutorial and theoretical treatments [13, 14, 17], cost-function and solver formulations [16], and open reference implementations [15]. Its cost is computational: an optimization must be solved within each control

period, which constrains horizon length and solver choice — a real-time-feasibility concern this report addresses directly (Chapter 7).

2.2.4 Nonlinear MPC and beyond

Linear MPC uses a linearized plant. Nonlinear MPC (NMPC) instead optimizes over the full nonlinear dynamics, enabling aggressive, large-envelope manoeuvres [18] at substantially higher computational cost and with weaker guarantees. For landing near hover — small angles, modest velocities — the linearization error is small, and linear MPC captures most of the benefit at a fraction of the cost.

2.2.5 Robust and sliding-mode control

A parallel line of work prioritizes robustness to disturbance and model uncertainty over nominal optimality. Sliding-mode control (SMC) drives the system onto a designed sliding surface and holds it there, giving strong invariance to matched disturbances; its classic drawback is control chattering, which higher-order and boundary-layer variants mitigate. Robust-control formulations (e.g. H_∞ and disturbance-observer designs) have been applied to moving-platform landing to guarantee performance under wind and platform-motion disturbances [19]. These methods are powerful where uncertainty dominates, but they are a different design philosophy from the optimal/predictive line this work follows, and they do not share a common cost structure that would make them directly comparable on equal terms (see Section 2.3).

2.2.6 Learning-based control

Reinforcement-learning and imitation approaches learn a landing policy from data, potentially capturing dynamics and disturbances that are hard to model. They remain data-hungry, harder to certify, and subject to a sim-to-real gap, and are outside the scope of a controlled, model-based comparative study.

2.2.7 Reviews

Broader surveys of quadrotor modelling, estimation, and control strategy place these method families in context and are a useful entry point to the wider literature [?].

2.3 Positioning of This Work

This report undertakes a comparative study of three controllers — PID, LQR/LQG, and MPC — for vision-guided precision landing. The selection of these particular methods, together with the exclusion of others surveyed above, follows from methodological considerations that are set out in this section.

2.3.1 Rationale for the selected controllers

The three chosen controllers form a progression of increasing capability along a single conceptual axis: the extent to which the control law exploits a model of the plant and a prediction of the future. PID is a reactive, model-free law; LQR/LQG is model-based and optimal in a quadratic sense, yet remains constraint-agnostic and reactive with respect to a time-varying reference; MPC augments the optimal formulation with precisely the two capabilities the preceding methods lack, namely explicit constraint enforcement and reference preview over a finite horizon. The comparison is therefore not merely a ranking of three algorithms but an attempt to isolate the contribution of each additional capability. Each transition in the progression corresponds

to a falsifiable hypothesis: whether optimality yields a measurable improvement over a hand-tuned PID law, and whether predictive preview yields a measurable improvement over optimal feedback when tracking a moving reference. This escalating-capability structure constitutes the organizing principle of the report.

2.3.2 The shared linear model and comparability

The comparison is grounded in a common foundation. The LQR and MPC formulations are constructed from the same linear plant model, the same quadratic cost weights, and the same state definition; the MPC terminal cost is taken as the solution of the LQR Riccati equation, such that an infinite-horizon MPC formally recovers the LQR control law. This shared basis is what renders the comparison fair in a precise sense: observed differences in closed-loop behaviour are attributable to constraint handling and predictive preview rather than to differences in the underlying model or its tuning. The same requirement explains why controllers drawn from a different model class are not admitted to the direct comparison.

2.3.3 Scope of the excluded methods

Nonlinear MPC, sliding-mode control, and robust control are reviewed above but are not included in the comparative study, for two reasons. The first is methodological: these methods do not share the linear model and quadratic objective on which the PID/LQR/MPC comparison rests, and their inclusion would confound the effect of the control law with that of a differing model class and cost structure, thereby undermining the fair-comparison principle that is central to this work. The second is practical: within the scope and duration of this internship, a rigorously conducted three-way comparison offers greater value than a broader study of lower depth. Learning-based control is excluded as outside the scope of a model-based investigation. These are decisions of scope rather than judgements of relative merit; for landing scenarios dominated by disturbance or requiring a large flight envelope, sliding-mode, robust, or nonlinear predictive control may be preferable, and these constitute natural directions for extending the present study.

2.3.4 Relation to the moving-platform literature

A substantial portion of the moving-platform landing literature adopts robust or predictive control [19, 12], motivated by the difficulty that reactive control encounters when the reference is in motion. The present work is consistent with this position but restricts its empirical contribution to the static-platform comparison and to the formulation of the moving-platform case. It thereby establishes the linear comparative foundation upon which a subsequent moving-platform study — via the moving-reference MPC formulation and the analysis of the steady-state lag exhibited by LQR under ramp tracking — follows as the natural next stage. So framed, the contribution is a rigorous foundation rather than an overstated moving-platform result.

3 System Architecture and Simulation Environment

This chapter describes the hardware and software architecture developed for vision-based precision landing of a multirotor UAV on a stationary or moving UGV platform. The complete system integrates onboard sensing, companion computing, offboard control, and the PX4 autopilot through a ROS 2-based communication framework. A simulation environment based on PX4 SITL and Gazebo was also developed to validate the proposed controllers before hardware deployment.

3.1 Hardware Platform Overview

The experimental platform consists of two robotic systems: a Holybro S500 quadrotor UAV and a skid-steer UGV serving as the landing platform. The UAV is equipped with a Pixhawk 6C flight controller, Jetson Orin Nano companion computer, downward-facing monocular camera, GPS module, ELRS receiver, and a front-facing depth camera reserved for future GPS-denied navigation experiments. The UGV carries a Jetson AGX companion computer together with LiDAR, IMU, and depth camera sensors. An ArUco marker is rigidly mounted on the top surface of the UGV to act as the visual landing target.

Power is supplied using a 4-cell LiPo battery. The propulsion system is powered directly through the battery, while a DC–DC converter provides a regulated supply for the companion computer. Peripheral devices including the Pixhawk, GPS module, ELRS receiver, and onboard cameras receive power through the onboard electronics. Figure 3.1 illustrates the overall hardware power distribution.

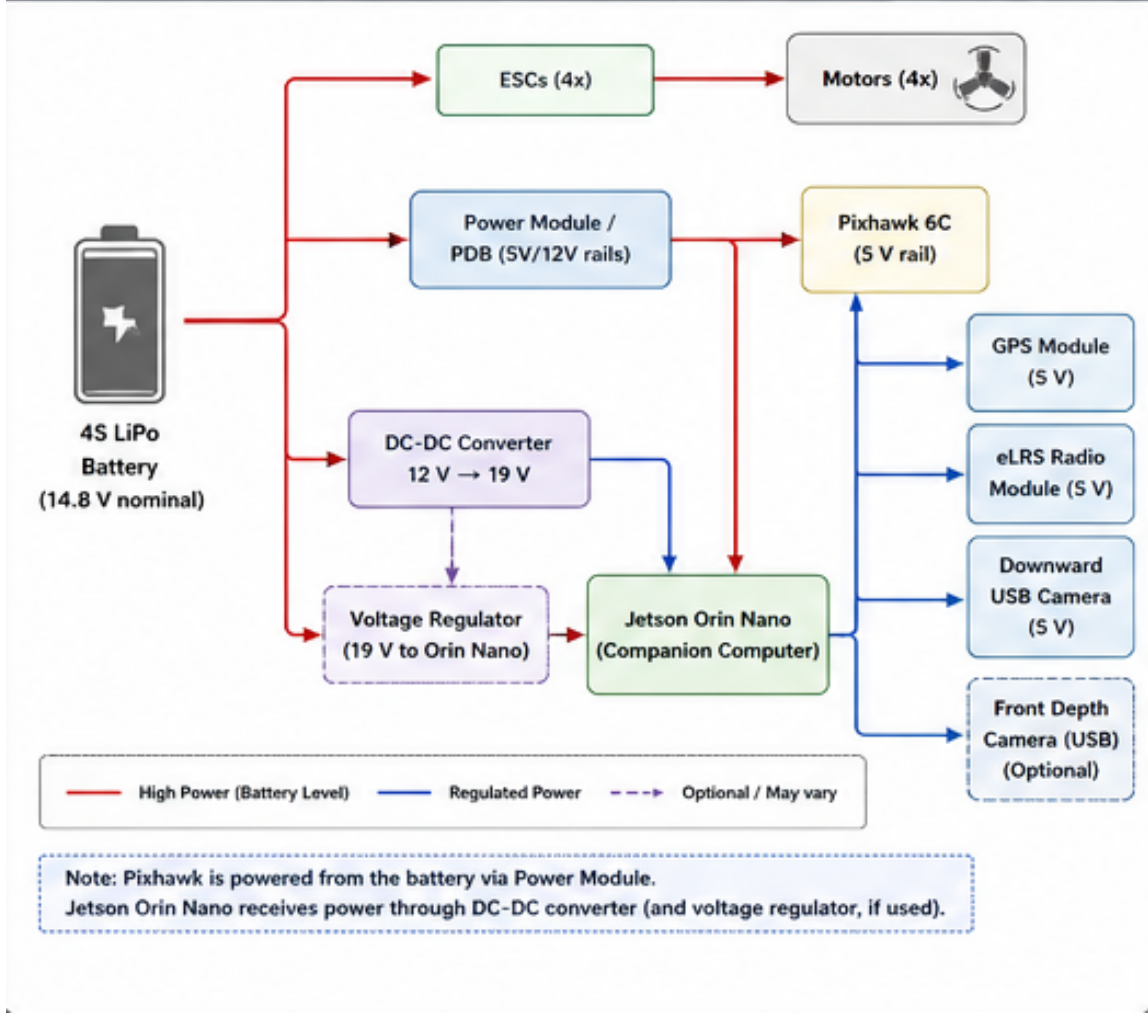
The separation of high-power propulsion circuitry from the computing subsystem improves electrical reliability while enabling simultaneous operation of onboard perception, communication, and flight control modules.

3.2 Software Stack

The software architecture is built upon PX4 Autopilot, ROS 2 Humble, Gazebo, and OpenCV. PX4 executes the low-level flight control algorithms, sensor fusion, and state estimation, while ROS 2 hosts the vision processing and high-level control nodes running on the companion computer. Gazebo provides a realistic simulation environment for software-in-the-loop validation prior to hardware implementation.

Visual perception is implemented using OpenCV, where ArUco marker detection and pose estimation provide the relative position of the landing target. Communication between PX4 and ROS 2 is established using the Micro XRCE-DDS bridge, allowing bidirectional exchange of vehicle states and control commands. QGroundControl is used for vehicle configuration, telemetry monitoring, parameter tuning, and flight supervision throughout both simulation and hardware experiments.

The modular software stack enables independent development and testing of perception, control, and communication components while maintaining compatibility with the PX4 offboard control framework.



commands to align the UAV with the landing target.

Figure 3.2 summarizes the complete vision-based pose estimation pipeline.

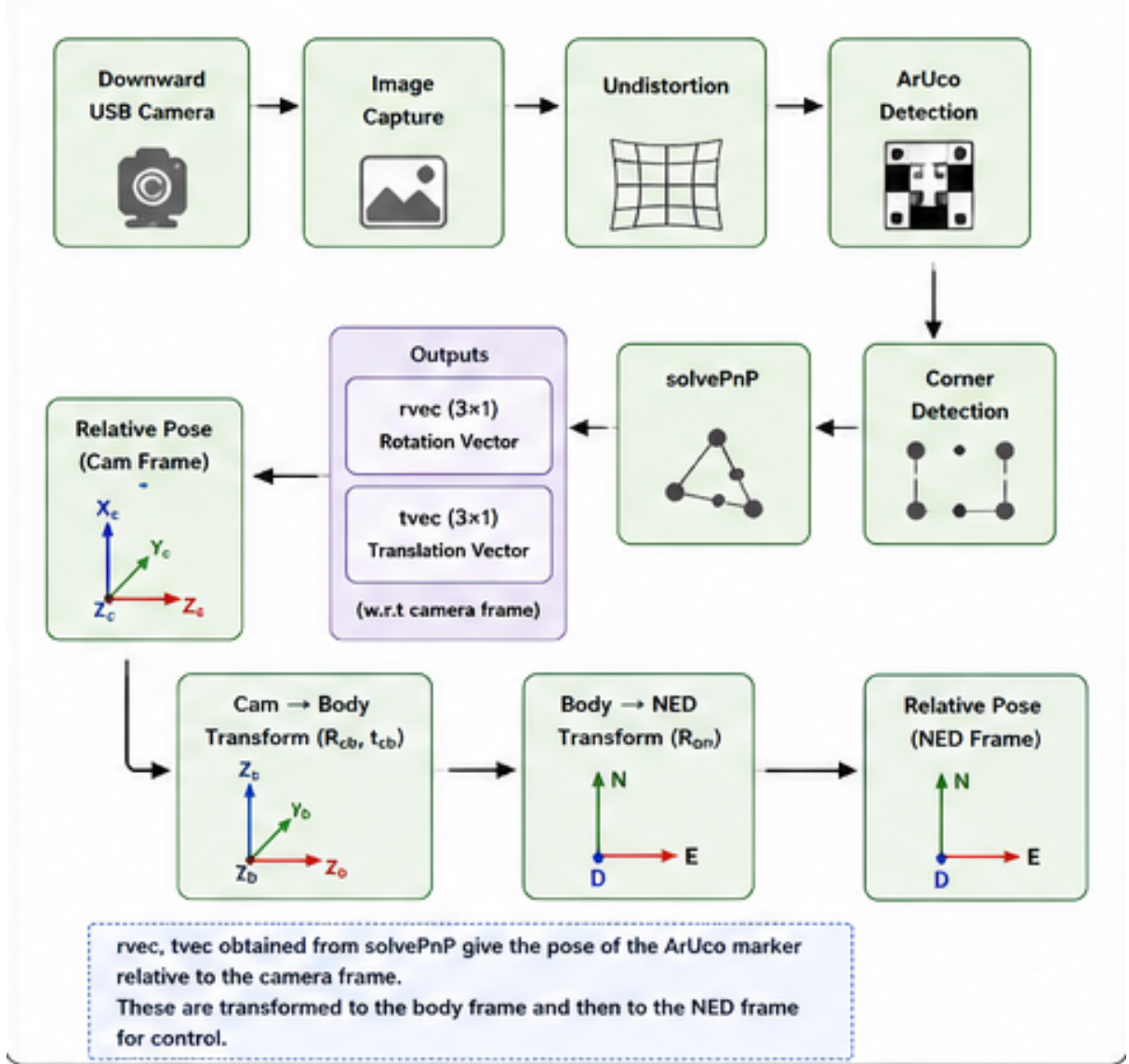


Figure 3.2: Vision pipeline for ArUco-based relative pose estimation and coordinate transformations.

3.4 Communication and Offboard Control Interface

Communication between the onboard companion computer and the PX4 flight controller is established through the Micro XRCE-DDS middleware over a serial UART connection via the TELEM2 port operating at a baud rate of 921600 bps. The Micro XRCE-DDS Client executes on the Pixhawk, while the Micro XRCE-DDS Agent runs on the Jetson Orin Nano. This architecture bridges the internal PX4 uORB middleware with the ROS 2 ecosystem, enabling seamless bidirectional exchange of state information and control commands.

In addition to the companion computer interface, TELEM1 is connected to a telemetry transceiver for communication with the Ground Control Station (QGroundControl), while TELEM3 interfaces with the ELRS receiver for manual RC override and flight safety. The downward-facing camera streams image data directly to the Jetson companion computer, where all perception

and high-level control algorithms are executed. Figure 3.3 illustrates the complete hardware communication architecture.

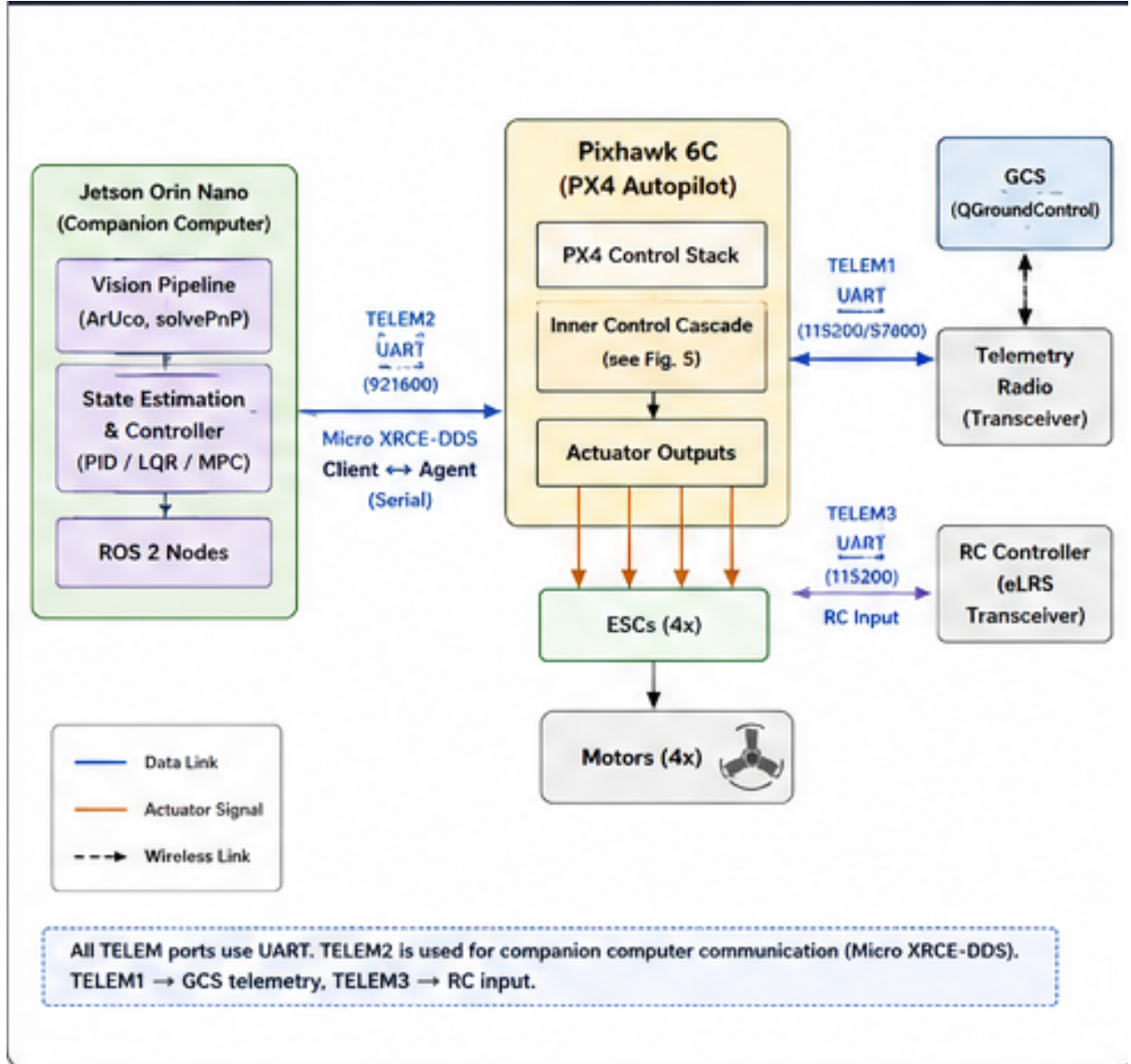


Figure 3.3: Hardware communication architecture showing sensor, telemetry and companion computer interfaces.

During offboard operation, ROS 2 nodes continuously publish `OffboardControlMode`, `TrajectorySetpoint`, and `VehicleCommand` messages through the Micro XRCE-DDS Agent. These commands are received by the PX4 Client, translated into uORB topics, and processed by the PX4 flight stack. Before switching to Offboard mode, PX4 verifies that valid offboard messages are being received continuously and that all preflight conditions are satisfied. If the command stream is interrupted during flight, the PX4 failsafe mechanism automatically exits Offboard mode and transitions the vehicle to a predefined safe flight mode.

The complete sequence of message exchange during Offboard operation is illustrated in Figure 3.4.

The generated trajectory setpoints are subsequently processed by the internal PX4 cascaded flight controller. Position errors are converted into velocity references, followed by attitude and body-rate regulation before actuator commands are generated for the Electronic Speed Controllers (ESCs) and motors. This hierarchical control structure separates high-level trajectory

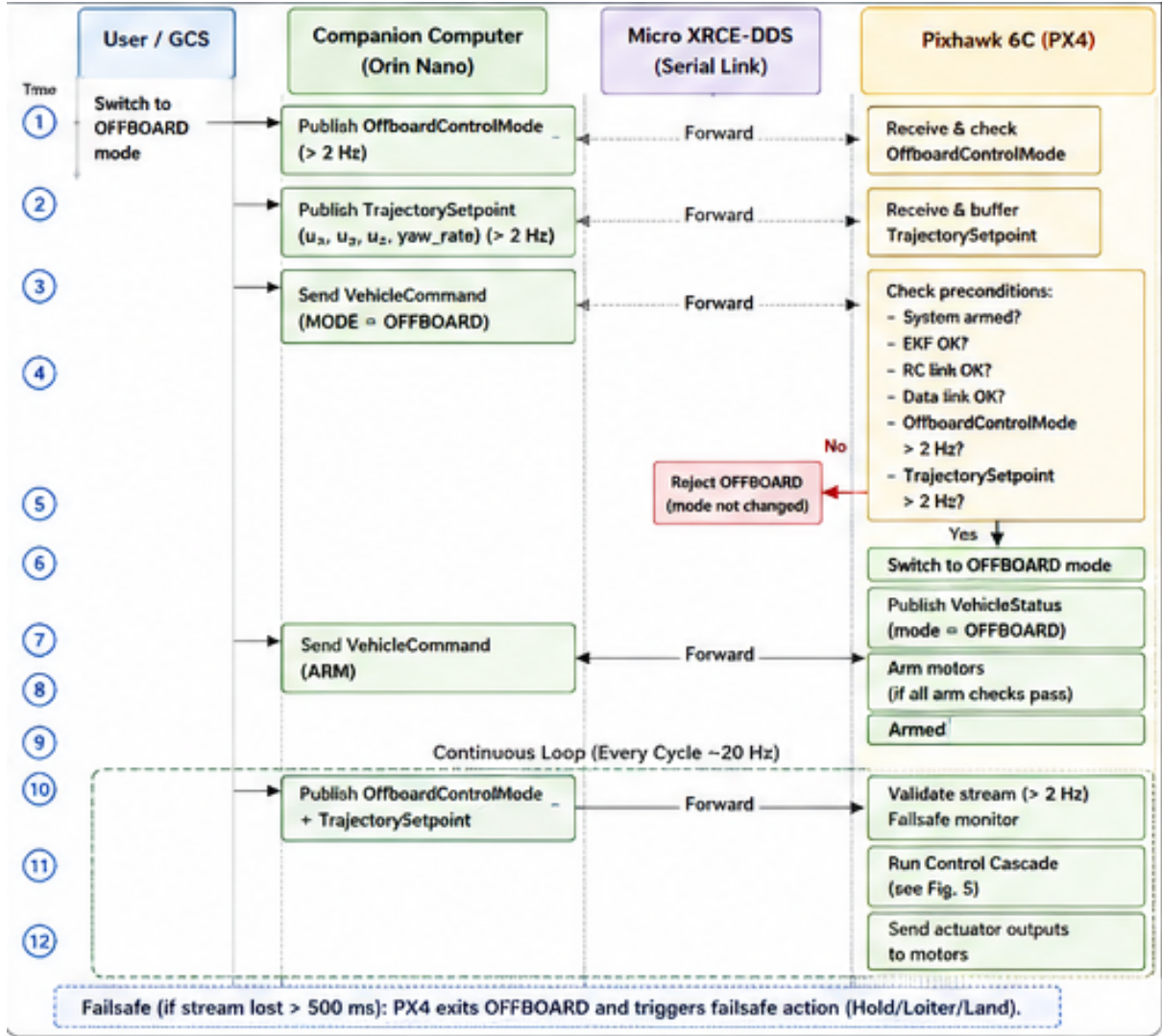


Figure 3.4: Sequence of data exchange during PX4 Offboard operation through the Micro XRCE-DDS bridge.

generation from low-level flight stabilization, allowing external controllers such as PID, LQR, and MPC to operate without modifying the embedded PX4 control loops.

Figure 3.5 summarizes the internal PX4 cascaded control architecture employed throughout this work.

3.5 Simulation Scenarios

A Software-in-the-Loop (SITL) simulation environment was developed using PX4 Autopilot, Gazebo, and ROS 2 to validate the proposed vision-based precision landing framework prior to hardware implementation. The simulated UAV executes the same PX4 flight stack used in hardware, while the perception and controller nodes run on ROS 2. A virtual downward-facing camera publishes image data to the vision pipeline, where ArUco detection and pose estimation are performed before generating velocity commands. These commands are transmitted to PX4 through the Micro XRCE-DDS interface, forming a closed-loop control system identical to the hardware implementation.

The overall software architecture employed during simulation is illustrated in Figure 3.6. The unified framework enables rapid controller development, parameter tuning, and repeatable

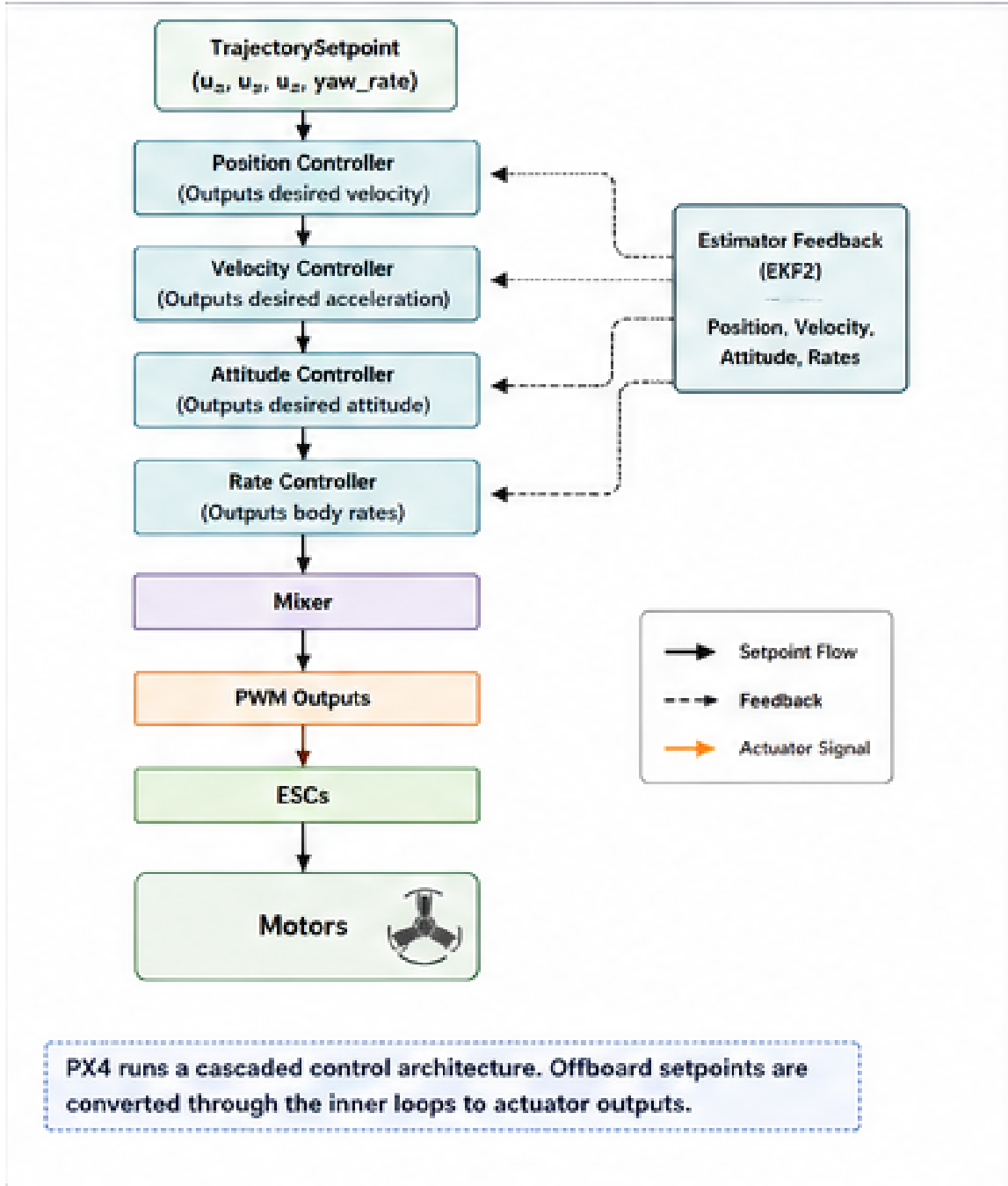


Figure 3.5: Internal cascaded flight control architecture implemented within PX4.

evaluation under identical operating conditions.

To emulate the intended application, a Gazebo world representing an electrical substation was created. The environment consists of a skid-steer UGV carrying an ArUco landing marker and a Holybro S500 UAV operating under PX4 SITL. Different viewpoints of the simulation world are shown in Figure 3.7, including the complete substation environment, top view of the landing platform, perspective view of the inspection scene, and the UAV during landing experiments. These scenarios were used to evaluate marker detection, relative pose estimation, trajectory tracking, and autonomous precision landing before hardware testing.

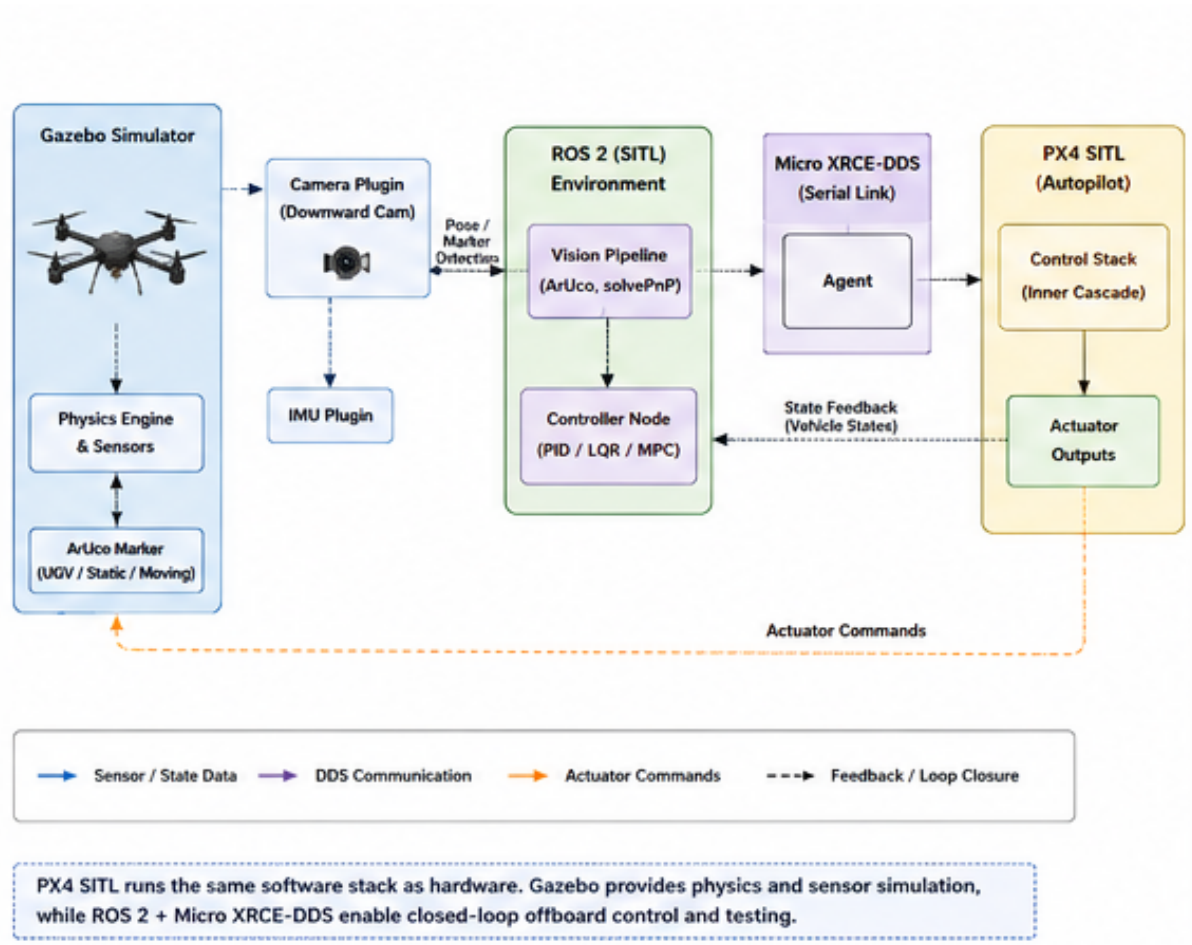
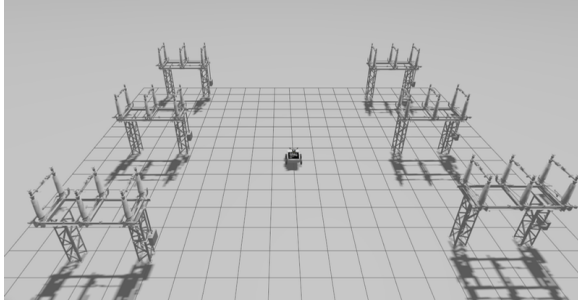
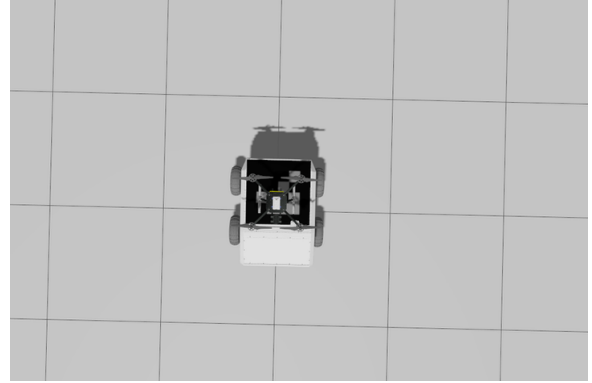


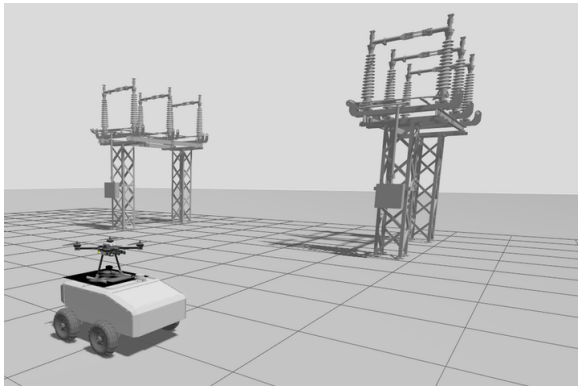
Figure 3.6: Software-in-the-Loop (SITL) architecture integrating Gazebo, ROS 2, PX4 Autopilot and the Micro XRCE-DDS communication bridge.



(a) Complete simulation world.



(b) Top view of the UGV with landing marker.



(c) Perspective view of the electrical substation environment.



(d) UAV during precision landing in simulation.

Figure 3.7: Representative Gazebo simulation scenarios used for controller development and validation.

4 UAV Dynamics and Plant Modeling

This chapter develops the control-oriented model used throughout the report. Starting from the full six-degree-of-freedom rigid-body dynamics, the model is reduced by hover linearization and axis decoupling to a pair of independent horizontal channels, the fast inner velocity loop is abstracted as a first-order lag, and the result is cast as a linear error-state system suitable for LQR and MPC design. All modelling assumptions are stated explicitly, and the controllability of the resulting system is verified.

4.1 Coordinate Frames and Conventions

Two frames are used: an inertial frame $\mathcal{I} = \{X_I, Y_I, Z_I\}$ and a body frame $\mathcal{B} = \{X_B, Y_B, Z_B\}$ fixed to the vehicle with origin at its centre of mass. A body-frame vector $P^{\mathcal{B}}$ maps to the inertial frame through

$$P^{\mathcal{I}} = P_{O_B}^{\mathcal{I}} + R_{\mathcal{B} \rightarrow \mathcal{I}} P^{\mathcal{B}}, \quad (4.1)$$

where $P_{O_B}^{\mathcal{I}}$ is the body-frame origin expressed in $\mathcal{I}$ and $R_{\mathcal{B} \rightarrow \mathcal{I}}$ is the rotation matrix for the Z - Y - X (yaw-pitch-roll) Euler sequence,

$$R_{\mathcal{B} \rightarrow \mathcal{I}} = R_z(\psi) R_y(\theta) R_x(\phi), \quad (4.2)$$

with ϕ, θ, ψ the roll, pitch, and yaw angles. The North-East-Down (NED) convention is adopted, so $+X$ is North, $+Y$ is East, and $+Z$ is Down. The velocity control loop operates in the inertial frame. Figure 4.1 illustrates the frame transformation and its realization on the vehicle used in this work.

4.2 Nonlinear Rigid-Body Dynamics

The translational motion is governed by two forces: the total motor thrust F_T , directed along the body $-Z_B$ axis, and gravity mg along $+Z_I$. Resolving in the inertial frame,

$$m \begin{bmatrix} \ddot{x}_I \\ \ddot{y}_I \\ \ddot{z}_I \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} + R_{\mathcal{B} \rightarrow \mathcal{I}} \begin{bmatrix} 0 \\ 0 \\ -F_T \end{bmatrix}. \quad (4.3)$$

Carrying out the multiplication yields the three translational equations of motion:

$$\ddot{x}_I = -\frac{F_T}{m} (\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi), \quad (4.4)$$

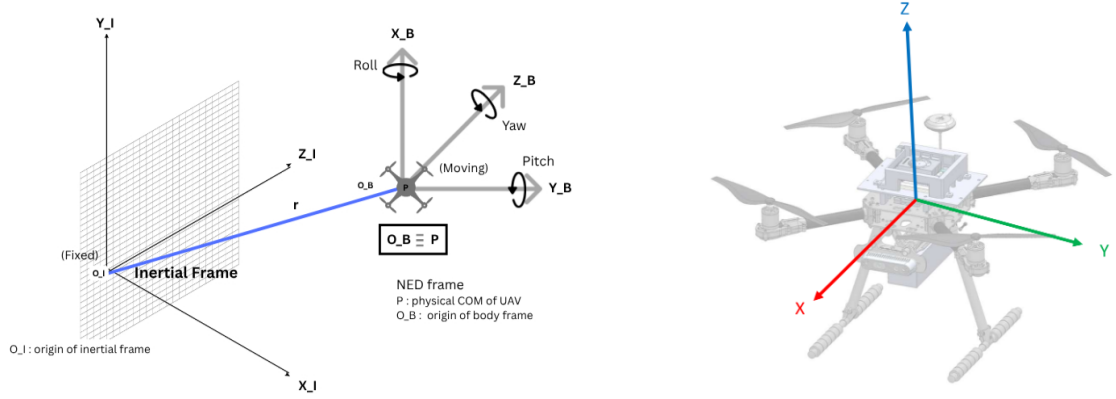
$$\ddot{y}_I = -\frac{F_T}{m} (\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi), \quad (4.5)$$

$$\ddot{z}_I = g - \frac{F_T}{m} (\cos \phi \cos \theta). \quad (4.6)$$

4.3 Hover Linearization and Axis Decoupling

The landing manoeuvre proceeds near hover at small attitude angles, which justifies the following linearization assumptions:

1. Near-hover thrust equilibrium: $F_T \approx mg$, hence $F_T/m \approx g$.
2. Small pitch: $\sin \theta \approx \theta$, $\cos \theta \approx 1$.



(a) Coordinate-frame transformation between the inertial frame $\mathcal{I}$ and the body frame $\mathcal{B}$.

(b) Body-frame axes on the quadrotor used in this work.

Figure 4.1: Coordinate frames. The body frame $\mathcal{B}$ is fixed to the vehicle with its origin at the centre of mass ($O_B \equiv P$); the rotation $R_{\mathcal{B} \rightarrow \mathcal{I}}$ maps body-frame quantities into the inertial frame $\mathcal{I}$ under the NED convention.

3. Small roll: $\sin \phi \approx \phi$, $\cos \phi \approx 1$.
4. Yaw held at the target: $\psi = 0$, hence $\sin \psi = 0$, $\cos \psi = 1$.
5. Products of small angles are neglected ($\phi\theta \approx \phi\psi \approx \theta\psi \approx 0$).
6. The vertical axis is decoupled and regulated separately ($\ddot{z}_I \approx 0$).

Applying these reductions collapses the horizontal dynamics to two independent, linear channels:

$$\ddot{x}_I \approx -g\theta, \quad \ddot{y}_I \approx g\phi, \quad (4.7)$$

or equivalently, with $v_x = \dot{x}_I$ and $v_y = \dot{y}_I$,

$$\dot{v}_x = -g\theta, \quad \dot{v}_y = g\phi. \quad (4.8)$$

The pitch axis drives North motion and the roll axis drives East motion; the two are symmetric and are treated identically hereafter.

4.4 Inner-Loop Abstraction: First-Order Velocity Lag

The high-level controllers designed in this report command velocity setpoints to the PX4 autopilot, which realizes them through its own inner control loops. Rather than model those loops in detail, their aggregate closed-loop response is abstracted. Assuming a high-bandwidth attitude loop, the commanded pitch is tracked essentially instantaneously, $\theta(t) \approx \theta_{\text{cmd}}(t)$, so from Eq. (4.8), $\dot{v}_x(t) = -g\theta_{\text{cmd}}(t)$. PX4's velocity loop applies a proportional-integral law,

$$\theta_{\text{cmd}}(t) = -\left[K_p(u_x(t) - v_x(t)) + K_i \int_0^t (u_x(\tau) - v_x(\tau)) d\tau\right], \quad (4.9)$$

where u_x is the commanded velocity setpoint and v_x the actual velocity. Substituting and differentiating gives a second-order relation between u_x and v_x . Under two reductions — the setpoint u_x from the discrete guidance layer is piecewise constant ($\dot{u}_x = 0$), and the heavily overdamped inner loop makes the second-order transient negligible ($\ddot{v}_x \approx 0$) — this collapses to a first-order lag,

$$\tau \dot{v}_x(t) + v_x(t) = u_x(t) \implies \dot{v}_x(t) = -\frac{1}{\tau}v_x(t) + \frac{1}{\tau}u_x(t), \quad (4.10)$$

with time constant $\tau = K_p/K_i$. The lateral axis reduces identically:

$$\tau \dot{v}_y(t) + v_y(t) = u_y(t). \quad (4.11)$$

The velocity command u , not attitude, is thus the effective control input to the guidance model — a modelling choice that keeps the plant linear and low-order.

4.5 Time-Constant Identification

The single parameter τ is identified empirically from the vehicle's velocity step response in simulation. An important caveat applies: the apparent time constant is inflated when PX4's jerk and acceleration limiters are active, since rate limiting slows the measured response independently of the underlying loop dynamics. Relaxing these limits recovers the true first-order τ ; the identification here is performed with the limiters relaxed so that the constraint behaviour later attributed to the controllers is not confounded by the autopilot's guidance layer.

4.6 Error State-Space Formulation

Precision landing is a regulation problem in the position error relative to the marker. With the sign convention held throughout this report,

$$x_{\text{err}} = p_x^{\text{marker}} - p_x, \quad y_{\text{err}} = p_y^{\text{marker}} - p_y, \quad (4.12)$$

i.e. error = marker position – drone position. For a stationary marker, differentiating gives $\dot{x}_{\text{err}} = -v_x$ and, using Eq. (4.10),

$$\ddot{x}_{\text{err}} = -\dot{v}_x = -\frac{1}{\tau}\dot{x}_{\text{err}} - \frac{1}{\tau}u_x, \quad (4.13)$$

and symmetrically for the lateral axis. Defining the state and input vectors

$$\mathbf{x} = [x_{\text{err}} \quad \dot{x}_{\text{err}} \quad y_{\text{err}} \quad \dot{y}_{\text{err}}]^\top, \quad \mathbf{u} = [u_x \quad u_y]^\top, \quad (4.14)$$

the continuous-time model $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ is

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{1}{\tau} & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -\frac{1}{\tau} \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 & 0 \\ -\frac{1}{\tau} & 0 \\ 0 & 0 \\ 0 & -\frac{1}{\tau} \end{bmatrix} \mathbf{u}. \quad (4.15)$$

The controllers operate in discrete time at sample period T_s . Exact (zero-order-hold) discretization of Eq. (4.15) gives $\mathbf{x}(k+1) = A_d \mathbf{x}(k) + B_d \mathbf{u}(k)$ with

$$A_d = \begin{bmatrix} 1 & \tau(1 - e^{-T_s/\tau}) & 0 & 0 \\ 0 & e^{-T_s/\tau} & 0 & 0 \\ 0 & 0 & 1 & \tau(1 - e^{-T_s/\tau}) \\ 0 & 0 & 0 & e^{-T_s/\tau} \end{bmatrix}, \quad (4.16)$$

$$B_d = \begin{bmatrix} -T_s + \tau(1 - e^{-T_s/\tau}) & 0 \\ -(1 - e^{-T_s/\tau}) & 0 \\ 0 & -T_s + \tau(1 - e^{-T_s/\tau}) \\ 0 & -(1 - e^{-T_s/\tau}) \end{bmatrix}. \quad (4.17)$$

4.7 Controllability Analysis

Because the two axes are decoupled and identical, controllability can be assessed on a single-axis subsystem

$$A_x = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{1}{\tau} \end{bmatrix}, \quad B_x = \begin{bmatrix} 0 \\ -\frac{1}{\tau} \end{bmatrix}, \quad (4.18)$$

whose controllability matrix is

$$\mathcal{C}_x = [B_x \quad A_x B_x] = \begin{bmatrix} 0 & -\frac{1}{\tau} \\ -\frac{1}{\tau} & \frac{1}{\tau^2} \end{bmatrix}, \quad \det \mathcal{C}_x = -\frac{1}{\tau^2} \neq 0. \quad (4.19)$$

The single-axis pair is therefore controllable, and by the block-diagonal structure the full four-state system has $\text{rank } \mathcal{C} = 4$: the system is completely controllable. The open-loop A matrix has an eigenvalue at the origin (the position integrator), so the plant is not asymptotically stable on its own; this is precisely why state feedback is required to drive the error to zero, and it also has a consequence for PID tuning discussed in Chapter 6 (the free integrator rules out open-loop Ziegler–Nichols).

4.8 Modelling Assumptions and Neglected Effects

The plant of Eq. (4.15) is deliberately minimal, and several assumptions carry weight. The hover linearization is valid only for small angles and modest velocities, as holds during a controlled descent but not for aggressive manoeuvres. The first-order inner-loop abstraction replaces PX4’s true higher-order attitude and velocity dynamics with a single lag; this is a structural approximation, and its imperfect fit is one source of the estimator model mismatch examined in Chapter 5. Axis decoupling assumes the yaw is regulated to zero and cross-axis coupling is negligible. Aerodynamic drag, ground effect during final descent, and wind disturbance are not modelled. These simplifications are what make the plant linear, low-order, and identical across all three controllers — the precondition for the fair comparison that follows — and their limitations are revisited in the discussion of results.

5 State Estimation

The controllers of the following chapters require the full error state — position *and* velocity relative to the marker. The vision pipeline, however, measures only relative *position*: an ArUco detection followed by a perspective-n-point solve returns the metric position of the marker in the camera frame, with no direct velocity measurement. A state estimator is therefore required to infer velocity from the position measurements and the known input, and to smooth the measurement noise. This chapter designs a Kalman filter for this purpose, selects its noise parameters from measured data, and assesses its statistical consistency. The role of the estimator within the comparative study is stated precisely at the end.

5.1 Motivation for Filtering

Two facts make filtering necessary rather than optional. First, the state is only partially measured: velocity must be reconstructed, and the simplest alternative — finite-differencing the position measurements — amplifies measurement noise unacceptably at the sample rate used here. Second, the ArUco position measurement is itself noisy, and feeding raw measurements to a feedback controller would inject that noise directly into the commanded velocity. A Kalman filter addresses both: it fuses the process model of Chapter 4 with the position measurements to produce a smoothed estimate of the complete state, including the unmeasured velocity.

5.2 Kalman Filter Design

The filter is built on the same discrete error-state model derived in Chapter 4, $\mathbf{x}(k+1) = A_d \mathbf{x}(k) + B_d \mathbf{u}(k)$, with state $\mathbf{x} = [x_{\text{err}}, \dot{x}_{\text{err}}, y_{\text{err}}, \dot{y}_{\text{err}}]^\top$. The measurement is the relative position recovered by the vision pipeline,

$$\mathbf{z}(k) = C \mathbf{x}(k) + \mathbf{w}(k), \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad (5.1)$$

so that only x_{err} and y_{err} are observed; the velocity states are inferred through the model. The standard predict–update recursion is

$$\text{Predict: } \hat{\mathbf{x}}(k|k-1) = A_d \hat{\mathbf{x}}(k-1|k-1) + B_d \mathbf{u}(k-1), \quad (5.2)$$

$$P(k|k-1) = A_d P(k-1|k-1) A_d^\top + Q, \quad (5.3)$$

$$\text{Update: } \boldsymbol{\nu}(k) = \mathbf{z}(k) - C \hat{\mathbf{x}}(k|k-1), \quad (5.4)$$

$$S(k) = C P(k|k-1) C^\top + R, \quad (5.5)$$

$$K(k) = P(k|k-1) C^\top S(k)^{-1}, \quad (5.6)$$

$$\hat{\mathbf{x}}(k|k) = \hat{\mathbf{x}}(k|k-1) + K(k) \boldsymbol{\nu}(k), \quad (5.7)$$

$$P(k|k) = (I - K(k) C) P(k|k-1), \quad (5.8)$$

where $\boldsymbol{\nu}(k)$ is the innovation, $S(k)$ its covariance, Q the process-noise covariance, and R the measurement-noise covariance. The prediction step runs every control tick using the last commanded input; the update step is applied whenever a marker detection is available. Figure 5.1 shows the resulting velocity estimate tracking the true (PX4) velocity, confirming that the unmeasured velocity is recovered even though it is never directly observed.

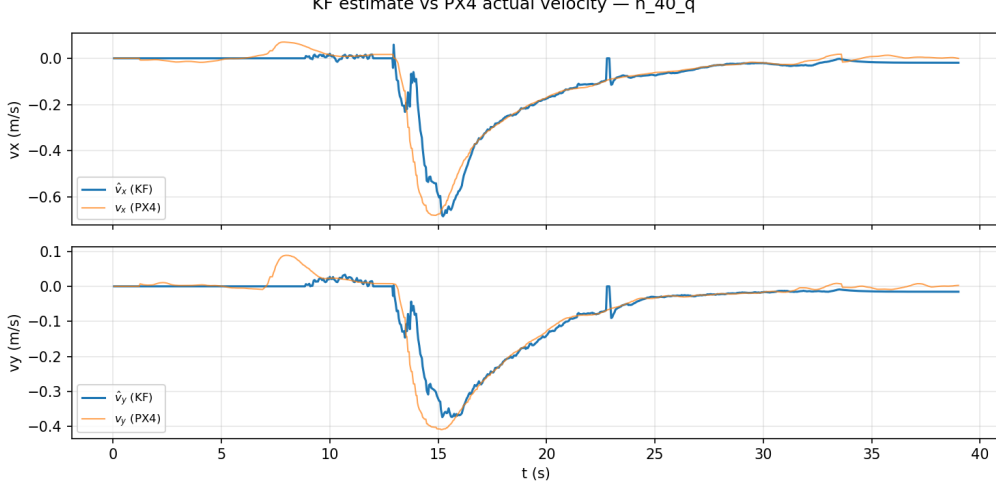


Figure 5.1: Kalman-filter velocity estimate ($\hat{v}_x, \hat{v}_y$) against the true PX4 velocity. Velocity is not measured; it is inferred through model propagation, and the estimate follows the truth closely.

5.3 Process and Measurement Noise Selection

Measurement noise R . Rather than treat R as a free tuning knob, it was measured directly. With the vehicle stationary above the marker, the variance of the ArUco-derived relative position was computed from a batch of detections, giving

$$R = \text{diag}(7.27 \times 10^{-4}, 5.08 \times 10^{-4}) \text{ m}^2, \quad (5.9)$$

estimated from roughly 810 samples at an operating range near 3 m. An earlier value estimated from higher-altitude data was found unrepresentative and was discarded in favour of this measured value.

Process noise Q . The process noise reflects confidence in the model. The velocity-state entries of Q were bounded physically by the unmodeled acceleration the first-order lag cannot represent, of order $(a_{\text{unmodeled}} T_s)^2$, and the position entries kept small. A sweep over candidate Q diagonals was performed and evaluated by the consistency test of Section 5.4; representative candidates included $\text{diag}(10^{-3}, 10^{-3}, 5 \times 10^{-2}, 5 \times 10^{-2})$ and $\text{diag}(10^{-4}, 10^{-4}, 10^{-3}, 10^{-3})$.

5.4 Consistency Validation via NIS

Filter consistency was assessed using the Normalized Innovation Squared (NIS) statistic,

$$\varepsilon(k) = \boldsymbol{\nu}(k)^\top S(k)^{-1} \boldsymbol{\nu}(k). \quad (5.10)$$

For a consistent filter, $\varepsilon(k)$ is χ^2 -distributed with degrees of freedom equal to the measurement dimension, here $n_z = 2$; the expected value is therefore $\mathbb{E}[\varepsilon] = 2$, and roughly 95% of samples should fall within the two-sided χ^2_2 interval $[0.05, 7.38]$.

The measured NIS (Fig. 5.2) has a mean substantially below the target of 2 — of order 0.2–0.4 across the tuning sweep — with fewer samples inside the 95% band than the nominal 95%. Interpreted honestly, this indicates that the filter is *conservative* rather than ideally consistent: the modelled innovation covariance S is larger than the innovations actually observed, so the filter reports more uncertainty than it exhibits. It is important to note what this is *not* — the filter does not diverge, the innovations remain small and unbiased, and the state estimate tracks the truth (Fig. 5.1).

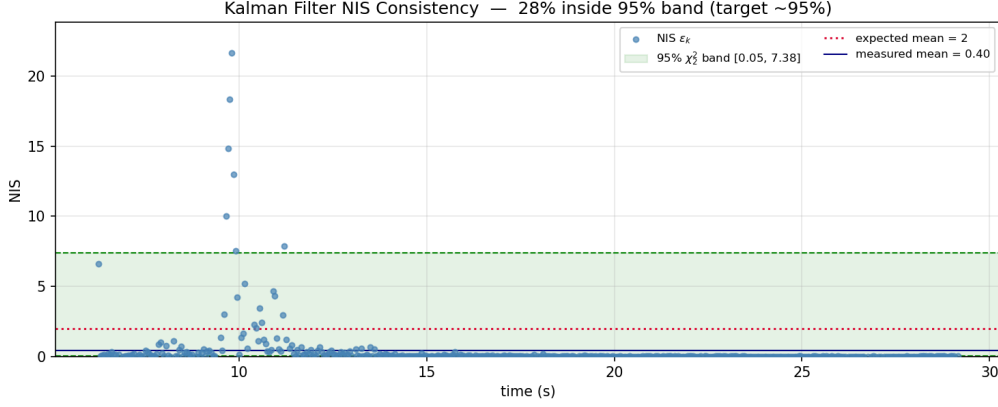


Figure 5.2: NIS consistency check with the measured R . The shaded region is the 95% χ^2_2 band $[0.05, 7.38]$; the dotted line marks the expected mean of 2. The measured mean lies well below 2, indicating a conservative filter.

The sub-target mean is attributable to **structural model mismatch** rather than to a mis-set scalar. The process model is a first-order lag standing in for PX4’s true higher-order attitude and velocity dynamics (Chapter 4); to prevent the predicted covariance from becoming over-confident against this imperfect model, the process noise Q must be kept elevated, which in turn inflates S and depresses the NIS. This is a genuine and reportable engineering finding: it locates the limitation in the fidelity of the low-order model, not in a tuning error that a different scalar would fix. Achieving a strictly consistent ($\mathbb{E}[\varepsilon] \approx 2$) filter would require a higher-fidelity process model, which is deferred as future work.

5.5 The Separation Principle and LQG

Combining the Kalman filter with the optimal LQR state-feedback gain of Chapter 6 yields a Linear Quadratic Gaussian (LQG) controller. The separation principle guarantees that, for the linear-Gaussian problem, the optimal estimator and the optimal regulator can be designed independently and combined without loss of optimality: the LQR gain is applied to the Kalman estimate in place of the true state. This is the precise sense in which $LQR + Kalman\ filter = LQG$.

5.6 Role of the Estimator in the Comparative Study

A methodological point must be made explicit, because it protects the fairness of the controller comparison. The estimator introduces its own dynamics and its own imperfections — as Section 5.4 shows, the filter here is conservative. If one controller consumed Kalman estimates while another consumed a different state, any performance difference would be confounded by the estimator. To avoid this, the comparative runs of Chapter 7 feed *identical Gazebo ground-truth state* to all controllers, so that measured differences reflect the control law alone. The Kalman filter is validated and demonstrated separately, as the estimator of the vision-in-the-loop LQG configuration, rather than as a shared front-end to the comparison. This separation is a deliberate part of the comparison methodology introduced in Chapter 6. One consequence is noted for transparency and revisited in Chapter 7: where a controller’s reported error is computed from the estimated state rather than ground truth, that difference in error source is stated explicitly so it is not mistaken for a difference in control performance.

%=====

6 Controller Design

This chapter describes the three controllers compared in this report — PID, LQR/LQG, and MPC — all built on the linear error-state model of Chapter 4. It begins with the methodological rule that governs the comparison, then treats each controller in turn, and closes with the distinction in how they handle input constraints, which is central to the results that follow.

6.1 Comparison Methodology

The purpose of the comparison is to attribute performance differences to the control *law*, not to the model, the tuning, or the estimator. Three rules enforce this. First, all controllers use the identical discrete plant (A_d, B_d) of Chapter 4 and the identical state definition. Second, the optimal controllers share weighting: the MPC stage cost uses the same Q and R as the LQR, and the MPC terminal cost is the LQR Riccati solution, so the two differ only in constraint handling and prediction, not in their objective. Third, and most important for the comparison runs, every controller consumes identical Gazebo ground-truth state (Chapter 5), removing estimator asymmetry as a confound. This fair-comparison principle is load-bearing: without it, a faster or slower response could reflect a better estimator or a different weighting rather than a better control law.

6.2 PID Control

The baseline is a per-axis PID law on the relative position error. For the North axis,

$$u_x(t) = K_p x_{\text{err}}(t) + K_i \int_0^t x_{\text{err}}(\tau) d\tau + K_d \dot{x}_{\text{err}}(t), \quad (6.1)$$

with the lateral axis identical by symmetry. PID is the reactive reference point: it needs no explicit state-space model and cannot enforce constraints, but it is simple and robust for the static-marker regulation task.

Tuning by closed-loop Ziegler–Nichols. The gains were tuned using the closed-loop (ultimate-gain) Ziegler–Nichols method: the proportional gain is raised until the loop sustains steady oscillation at the ultimate gain K_u with period T_u , from which K_p , K_i , K_d follow by the standard Ziegler–Nichols table. The *open-loop* Ziegler–Nichols method is not applicable here: it presumes a self-regulating (S-shaped) step response, whereas the plant contains a free integrator (the position state, Chapter 4) and does not settle to a steady value open-loop. The finite latency of the vision pipeline supplies the phase lag that makes a sustained oscillation — and hence the closed-loop method — possible.

6.3 LQR and LQG

The Linear Quadratic Regulator computes the state-feedback gain minimizing the infinite-horizon quadratic cost

$$J = \sum_{k=0}^{\infty} \left(\mathbf{x}(k)^\top Q \mathbf{x}(k) + \mathbf{u}(k)^\top R \mathbf{u}(k) \right), \quad (6.2)$$

subject to $\mathbf{x}(k+1) = A_d \mathbf{x}(k) + B_d \mathbf{u}(k)$. The optimal control is the constant state feedback $\mathbf{u}(k) = -K \mathbf{x}(k)$, where $K = (R + B_d^\top P B_d)^{-1} B_d^\top P A_d$ and P solves the discrete algebraic

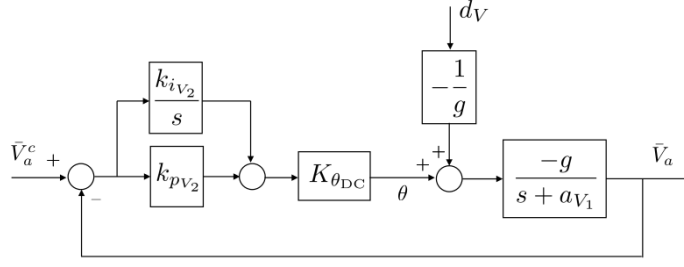


Figure 6.1: PX4 inner proportional-integral velocity loop, abstracted in Chapter 4 as a first-order lag with time constant τ .

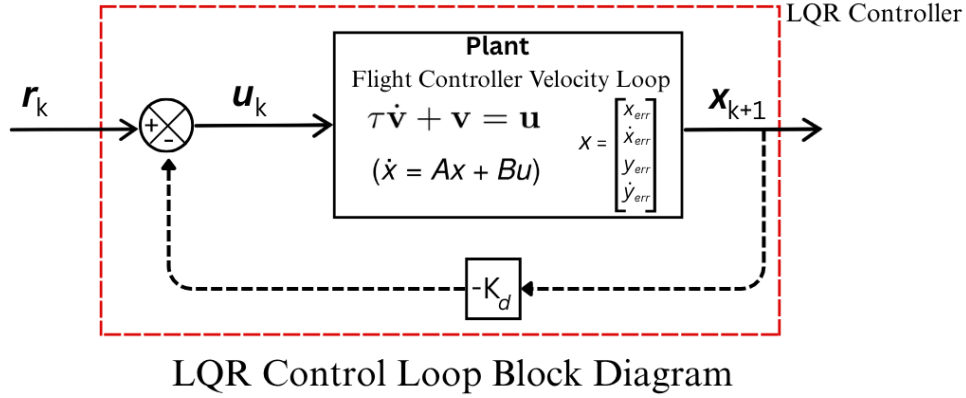


Figure 6.2: Discrete LQR guidance loop. The regulator gain $-K_d$ acts on the error state to produce the velocity setpoint passed to the inner loop.

Riccati equation (DARE)

$$P = A_d^\top P A_d - A_d^\top P B_d (R + B_d^\top P B_d)^{-1} B_d^\top P A_d + Q. \quad (6.3)$$

The weights Q and R trade tracking accuracy against control effort. Q was set to penalize position error primarily, and R was selected by a sweep, with $R = 5$ adopted as the comparison baseline; the effect of R is examined in Chapter 7.

Constraint handling by clipping. The LQR gain is unconstrained, so the commanded velocity is saturated (clipped) to the per-axis velocity cap before being sent to PX4. This is the only mechanism by which a plain LQR respects the input limit, and it is a post-hoc one — the optimizer is unaware of the cap. The contrast with MPC on exactly this point is developed in Section 6.5.

From LQR to LQG. When the state is estimated rather than measured, the LQR gain is applied to the Kalman estimate of Chapter 5; the resulting output-feedback controller is LQG, and the separation principle guarantees the estimator and regulator can be designed independently. The inner PX4 velocity loop and the guidance loop are shown in Figs. 6.1 and 6.2.

6.4 Model Predictive Control

MPC solves, at each control step, a finite-horizon optimal control problem over a horizon of N steps, applies the first input, and re-solves at the next step. The problem is

$$\min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \mathbf{x}_N^\top Q_f \mathbf{x}_N + \sum_{k=0}^{N-1} \left(\mathbf{x}_k^\top Q \mathbf{x}_k + \mathbf{u}_k^\top R \mathbf{u}_k \right), \quad (6.4)$$

subject to the dynamics $\mathbf{x}_{k+1} = A_d \mathbf{x}_k + B_d \mathbf{u}_k$ and the hard per-axis input constraint $|\mathbf{u}_k| \leq u_{\max}$. Two design choices are deliberate. First, the stage weights Q and R are those of the LQR (Section 6.1), and the terminal weight is the LQR Riccati matrix, $Q_f = P$ from Eq. (6.3): this makes the finite-horizon cost a truncation of the LQR's infinite-horizon cost, so that as $N \rightarrow \infty$ the MPC recovers the LQR law, and gives the short-horizon controller the horizon-independence examined in Chapter 7. Second, the problem is posed in *absolute* inputs (not input increments), avoiding a hidden integrator, with velocity-cap constraints only. The problem is a convex quadratic program, solved online with CasADi/Opti and IPOPT.

Moving reference. For a moving platform, the platform's motion enters only through the reference trajectory, never through A_d or B_d . The marker position is projected forward over the horizon at its estimated constant velocity,

$$\mathbf{x}_k^{\text{ref}} = \mathbf{p}^{\text{marker}} + \mathbf{v}^{\text{marker}} k T_s, \quad (6.5)$$

and the stage cost penalizes deviation from this moving reference. This is the mechanism by which MPC anticipates platform motion; it is developed in Chapter 8.

6.5 Constraint Handling: Clipping versus Hard Constraints

The controllers differ fundamentally in how they respect the velocity limit. PID and LQR produce an unconstrained command that is clipped afterwards: the control law optimizes as if no limit existed and the limit is imposed only at the output, so near saturation the realized behaviour departs from what the law intended. MPC, by contrast, embeds $|\mathbf{u}_k| \leq u_{\max}$ as a hard constraint inside the optimization, so every planned input is feasible by construction and the controller shapes its trajectory in anticipation of the limit. This distinction is not merely formal; it is visible in the results whenever the commanded velocity approaches the cap, and it is one of the two capabilities (with prediction) that separate MPC from LQR.

7 Results and Comparative Analysis: Static Marker

This chapter reports the static-marker results obtained in PX4 Software-in-the-Loop with Gazebo. A scoping note applies throughout. The *quantitative* comparison is drawn between LQR and MPC, which share the identical linear model, the identical weighting matrices Q and R , and the identical estimator (Chapters 4 and 5); this shared foundation is what makes their comparison fair in a precise sense. PID is presented as a *reactive baseline* and characterized on its own terms: because it does not share the optimal controllers’ cost structure and was tuned independently, comparing it numerically against the optimal controllers under a single table would conflate differences in the control law with differences in run conditions. The PID-to-LQR-to-MPC narrative is therefore read as a progression in capability, not as a single controlled numerical benchmark.

7.1 Metrics and Evaluation Protocol

Performance is characterized with the following quantities:

RMSE	Root-mean-square tracking error (m); trajectory-averaged, and dominated by the initial large offset.
IAE	Integral of the absolute error ($\text{m} \cdot \text{s}$).
ITAE	Integral of the time-weighted absolute error ($\text{m} \cdot \text{s}^2$); penalizes late error, so it captures convergence speed.
SSE	Steady-state error (m); mean error over the final portion of the run.
Settling time	Time to enter and remain within a 0.1 m band (s).
Touchdown error	Final relative position error at landing (m).
$\int u^2 dt$	Control effort; penalizes large commands.
$\text{TV}(u)$	Total variation of the command; penalizes non-smooth commands.

For MPC, solver time per step is additionally reported against the control-period deadline.

Confound, stated explicitly. Where a controller’s error is computed from the Kalman-estimated state rather than ground truth — as in the MPC runs, whose logs use $\hat{x}_{\text{err}}/\hat{y}_{\text{err}}$ — the error source differs from a ground-truth-referenced run, and the two are not strictly interchangeable. This is flagged wherever it applies so that a difference in error *source* is not read as a difference in control *performance*.

7.2 PID Results (Reactive Baseline)

The PID controller was tuned incrementally, adding the derivative and then the integral term to a proportional baseline. Figure 7.1 overlays the horizontal error for the three stages: proportional action sets the closing speed but leaves an oscillatory approach; derivative action damps the overshoot; integral action removes the residual steady offset and regulates the error to within the landing tolerance. Figure 7.2 isolates the settling phase ($t > 20\text{s}$), where the progressive reduction of oscillation from P to PD to PID is clearest. The tuning-sweep metrics are summarized in Fig. 7.3. These results establish PID as a competent reactive baseline on the static marker; they are reported qualitatively and are not entered into a numerical comparison with the optimal controllers, per the scoping note above.

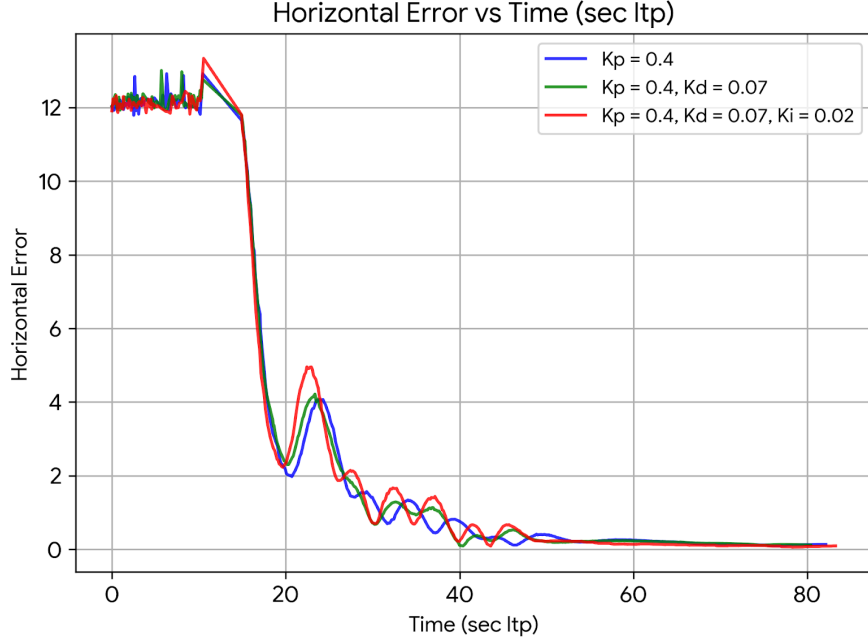


Figure 7.1: PID design study: horizontal error versus time for the proportional (P), proportional-derivative (PD), and full PID controllers. State source: Gazebo GT.

Table 7.1: LQR performance metrics for the control-effort weighting sweep, with $R = 5$ the adopted baseline. State source: Gazebo ground truth.

Metric	$R = 1$	$R = 5$ (baseline)	$R = 10$
RMSE (m)	1.2282	1.2557	0.9722
IAE (m · s)	19.1572	13.0922	15.8078
ITAE (m · s ²)	219.0133	129.1595	113.7922
SSE (m)	0.0912	0.0222	0.0921
Touchdown err. (m)	0.0912	0.0222	0.0921

RMSE: root-mean-square error; IAE: integral absolute error; ITAE: integral time-weighted absolute error; SSE: steady-state error.

7.3 LQR Results

The LQR uses the Bryson-rule weights of Chapter 6, refined by a sweep over the control-effort weight R with $R = 5$ adopted as the baseline. Smaller R yields faster convergence at higher effort and larger peak velocity; larger R trades convergence speed for smoother, lower-effort commands (Fig. 7.4). The quantitative comparison of the LQR baseline against MPC is presented in Section 7.5, where the two share Q and R .

The corresponding metrics for the R -sweep are given in Table 7.1.

7.4 MPC Results

The MPC was evaluated across prediction horizons $N \in \{10, 20, 40\}$, in two terminal-cost configurations: with the DARE terminal cost ($Q_f = P$, the infinite-horizon-equivalent) and without it ($Q_f = Q$).

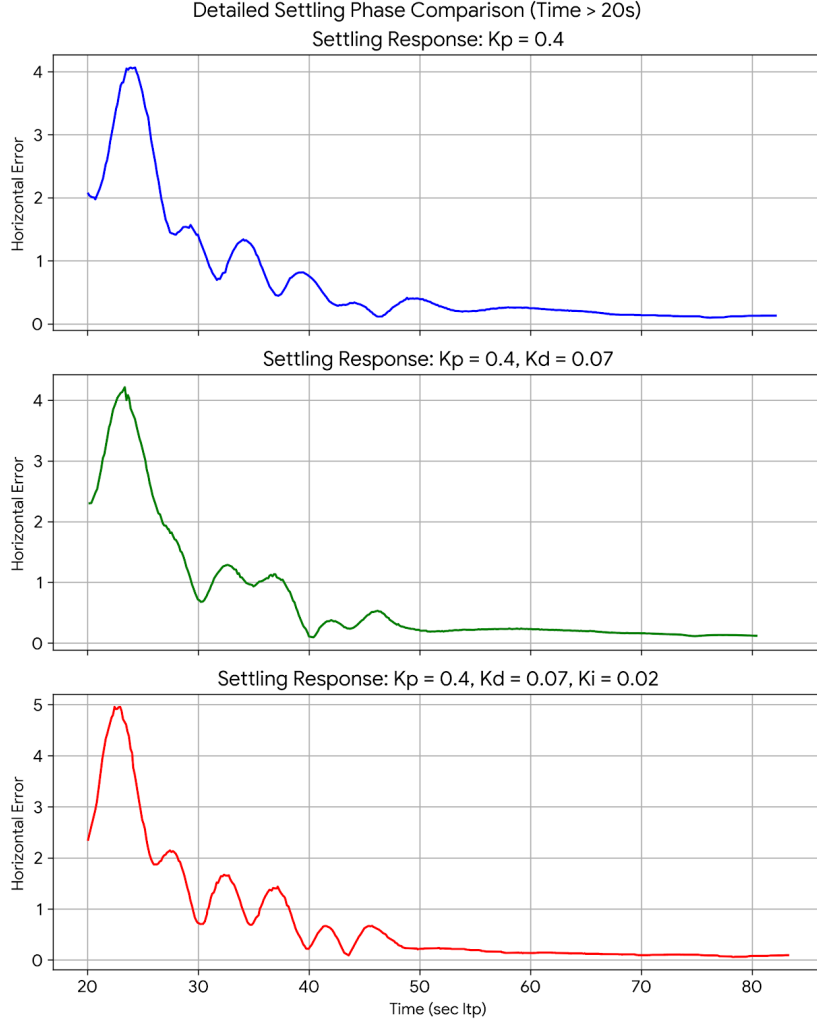


Figure 7.2: Detailed settling-phase comparison ($t > 20$ s) for the P, PD, and PID controllers. Derivative and integral action progressively suppress the residual oscillation. State source: Gazebo GT.

Horizon sensitivity. Figure 7.5 plots RMSE, steady-state error, and touchdown error against N for both configurations. Two trends are visible. First, accuracy generally improves with horizon length. Second, the terminal cost matters most at short horizons: with $Q_f = P$ the errors remain low and nearly flat across N , whereas with $Q_f = Q$ they grow markedly as N shrinks, because a short horizon without a terminal value function is myopic. The best-behaved configuration is the DARE terminal cost, which keeps a short, computationally cheap horizon performing comparably to a long one.

The exact values behind Fig. 7.5 are tabulated for both terminal-cost configurations in Tables 7.2 and 7.3.

Prediction versus realization. Figure 7.6 overlays, for each horizon, the sequence of planned horizons (dashed) against the realized error path (solid). The close agreement between planned and realized trajectories confirms that the model-based prediction is faithful; with $Q_f = P$ the paths are smooth and direct, whereas the short-horizon $Q_f = Q$ case shows the more meandering approach consistent with its poorer metrics.

Controller	e0	rise_time	settling_time	overshoot_x_pct	overshoot_y_pct	rmse_2d	ctrl_effort
Kp = 0.2	12.4	33.8	48.7	0	0	3.1	41.8
Kp = 0.4	11.5	15.2	25.9	8.4	55.9	2.1	38.1
Kp = 0.5	11.2	5.8	143.5	55.9	108	2.7	159
Kp = 0.04, Kd = 0.03	11.7	14.5	25	10.1	54.6	2.1	37.7
Kp = 0.04, Kd = 0.05	11.6	6	31.3	7	51.2	2.2	37.1
Kp = 0.04, Kd = 0.07	11.7	13.7	23.8	11.6	53.1	2.1	37.7
Kp = 0.04, Kd = 0.1	11.7	14	31	13.9	56.8	2.2	37.5
Kp = 0.04, Kd = 0.07, Ki = 0.01	11.7	14.5	32.2	14	61.2	2	39.4
Kp = 0.04, Kd = 0.07, Ki = 0.02	11.6	13.9	31.4	14.9	64.6	2.2	38.6
Kp = 0.04, Kd = 0.07, Ki = 0.03	11.3	12.2	27.4	6.4	46.8	1.8	31.4
Kp = 0.04, Kd = 0.07, Ki = 0.05	11.6	11.6	35	19.6	65.7	1.6	40.3

Figure 7.3: PID tuning-sweep metrics across proportional, derivative, and integral gain settings. Highlighted cells mark the best settling time, 2-D RMSE, and control effort in each group.

Table 7.2: Infinite-horizon-equivalent MPC ($Q_f = P$, $R = 5$): performance metrics versus prediction horizon N . Error metrics use the KF-estimated state. “—” denotes the 0.1 m settling band not reached within the run.

Metric	$N = 10$	$N = 20$	$N = 40$
RMSE (m)	1.79	1.50	1.46
IAE (m · s)	14.81	16.50	12.38
ITAE (m · s ²)	223.0	271.4	218.0
SSE (m)	0.087	0.032	0.081
Settling time (s)	—	13.48	13.30
Touchdown err. (m)	0.117	0.009	0.081

RMSE: root-mean-square error; IAE: integral absolute error; ITAE: integral time-weighted absolute error; SSE: steady-state error.

Realized descent paths. The realized three-dimensional descent paths (Fig. 7.7) show the same ordering: longer horizons and the DARE terminal cost produce straighter, better-damped approaches to the marker, while the shortest horizon without terminal cost wanders most before converging.

7.5 LQR versus MPC: Comparative Analysis

The fair quantitative comparison is between LQR and MPC, which share the model, the weights Q and R , and the estimator. Two views are reported.

Tracking quality. Figure 7.8 compares the Kalman-estimated velocity against the true PX4 velocity for the LQR ($R = 5$), the infinite-horizon MPC ($N = 20$, $Q_f = P$), and the short-horizon MPC ($N = 20$, $Q_f = Q$). The infinite-horizon MPC shows the closest alignment between estimate and truth, with the smallest overshoot and the fastest recovery; the short-horizon MPC has larger tracking error and slower recovery; and the LQR exhibits the largest transient mismatch and overshoot. This ranks the controllers exactly as their capability predicts, on identical weighting.

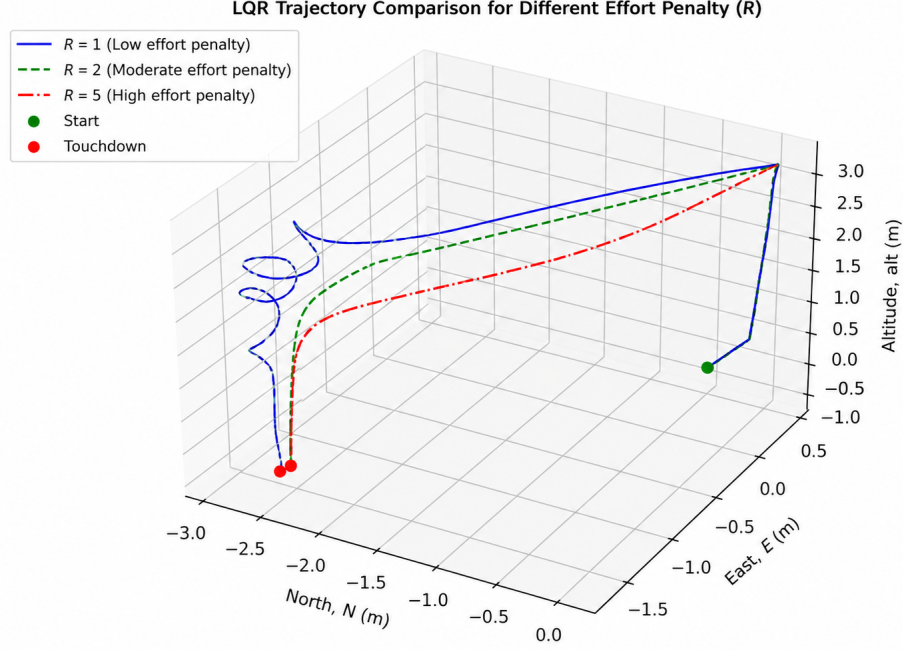


Figure 7.4: LQR control-effort weighting study: static-marker error response for a sweep of R , with $R = 5$ the adopted baseline. State source: Gazebo GT.

Table 7.3: Short-horizon MPC ($Q_f = Q$): the same metrics versus prediction horizon N . Error metrics use the KF-estimated state. “—” denotes the 0.1 m settling band not reached within the run.

Metric	$N = 10$	$N = 20$	$N = 40$
RMSE (m)	1.84	1.81	1.64
IAE (m · s)	135.93	48.55	21.33
ITAE (m · s ²)	5407.8	1150.5	369.8
SSE (m)	0.616	0.413	0.190
Touchdown err. (m)	0.501	0.319	0.139

RMSE: root-mean-square error; IAE: integral absolute error; ITAE: integral time-weighted absolute error; SSE: steady-state error.

Control effort. Figure 7.9 plots control effort $\int u^2 dt$ against horizon for both MPC configurations, with the LQR baseline marked as a single point. The comparison shows the effort–horizon trade: the DARE terminal cost commands more effort than the un-augmented short-horizon MPC (it acts more decisively toward the target), with both converging toward the LQR effort level as the horizon grows. Read together with the tracking-quality result, MPC achieves its tighter tracking at a controlled and comparable effort cost.

Q ualitatively, the reactive PID baseline (Section 7.2) also lands within tolerance on the static marker, but with the more oscillatory approach visible in Figs. 7.1–7.2. It is not entered into the quantitative LQR–MPC comparison, since it neither shares the optimal cost structure nor was run under matched conditions. The overall reading of the static-marker study is therefore that all three controllers can regulate to the marker, while the optimal and predictive controllers provide progressively smoother, better-damped, and (for MPC) constraint-aware approaches.

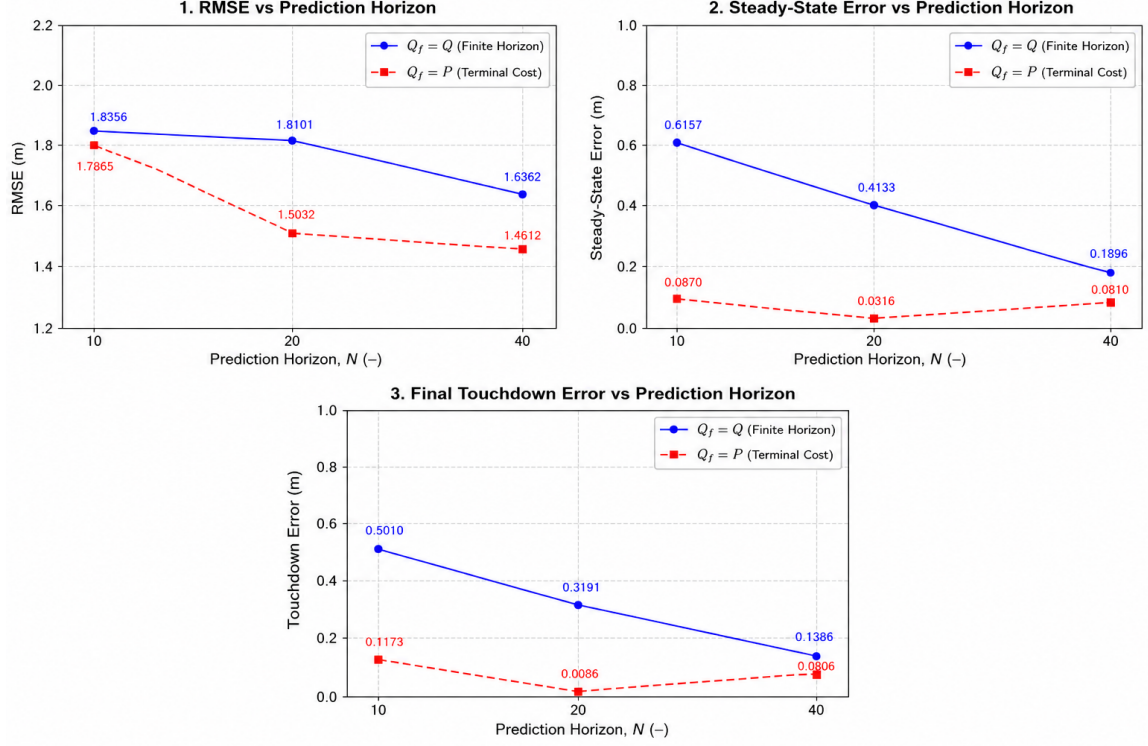


Figure 7.5: MPC accuracy versus prediction horizon N : RMSE, steady-state error, and final touchdown error, for $Q_f = Q$ (no terminal cost) and $Q_f = P$ (DARE terminal cost). Error metrics use the KF-estimated state.

Overall Result. PID established a stable baseline, LQR improved convergence with little computational overhead, and MPC delivered the best overall tracking while respecting velocity constraints. Increasing the prediction horizon beyond $N = 20$ produced only marginal improvements.

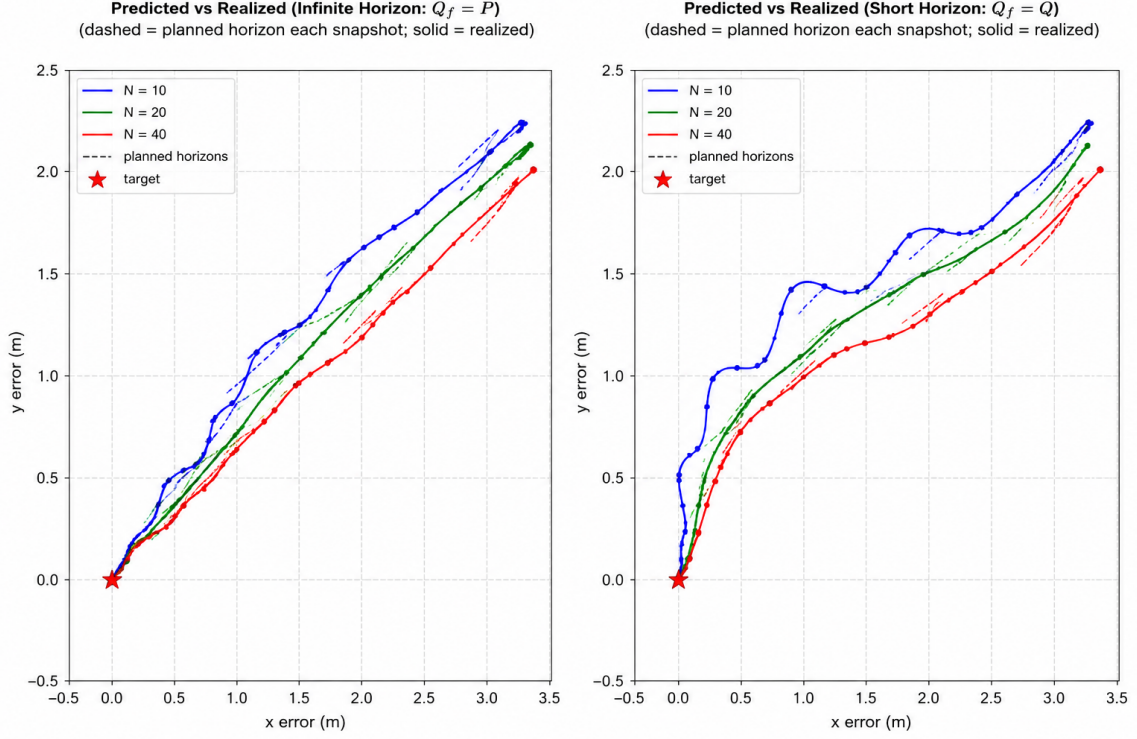


Figure 7.6: Predicted versus realized error paths for $N \in \{10, 20, 40\}$: DARE terminal cost $Q_f = P$ (left) and no terminal cost $Q_f = Q$ (right). Dashed lines are the per-snapshot planned horizons; solid lines are the realized path. State source: KF.

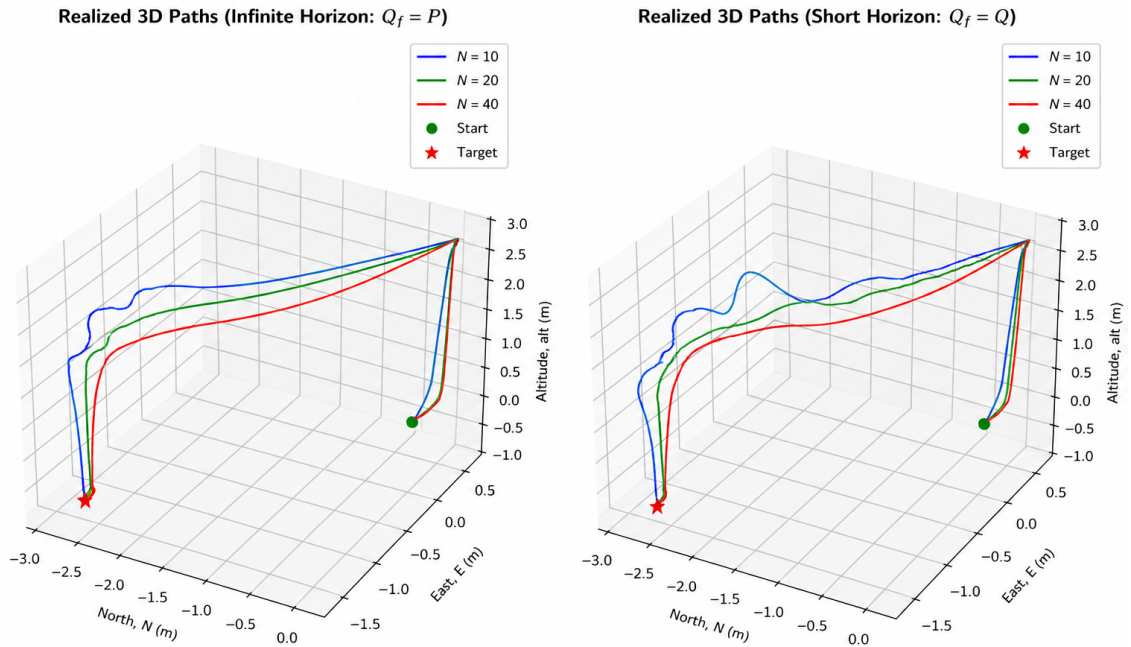


Figure 7.7: Realized 3-D descent paths for $N \in \{10, 20, 40\}$: $Q_f = P$ (left) and $Q_f = Q$ (right). Longer horizons and the DARE terminal cost yield straighter descents. State source: KF.

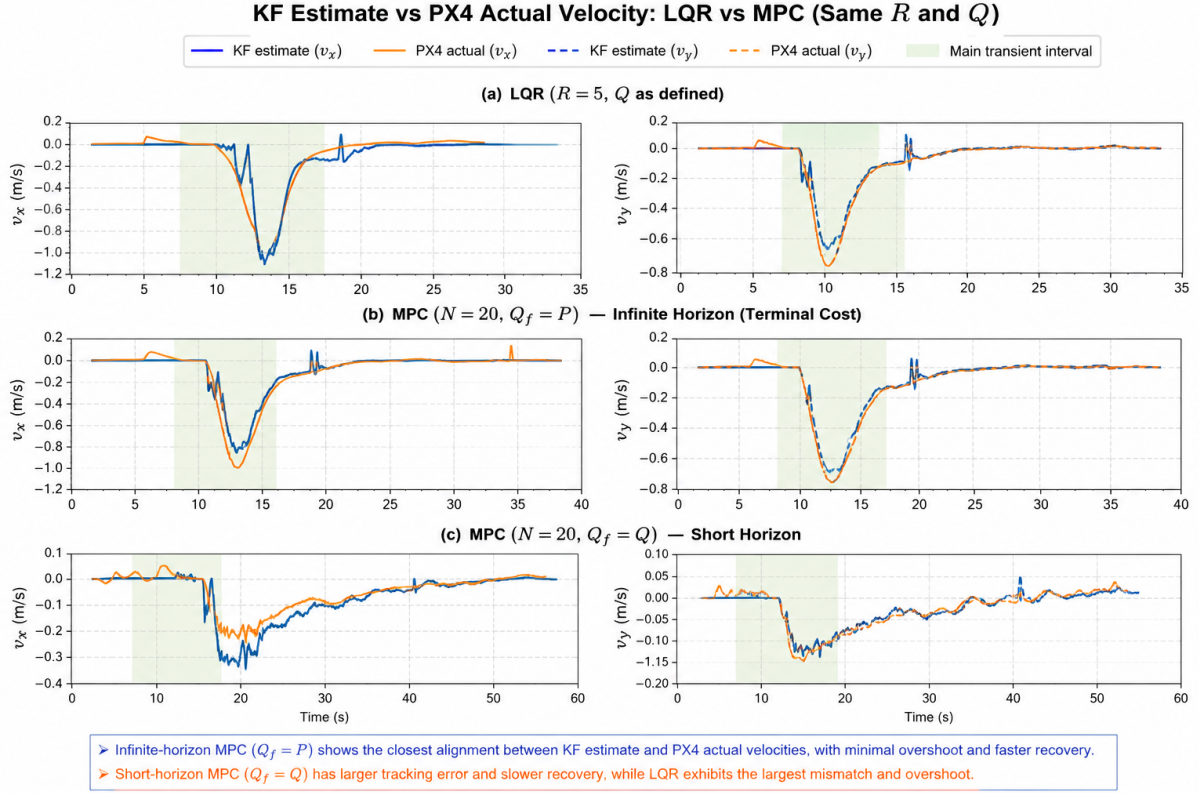


Figure 7.8: Kalman-estimate versus PX4-actual velocity for LQR and MPC under identical R and Q : (a) LQR $R = 5$; (b) MPC $N = 20$, $Q_f = P$; (c) MPC $N = 20$, $Q_f = Q$. The infinite-horizon MPC tracks most closely; LQR shows the largest mismatch.

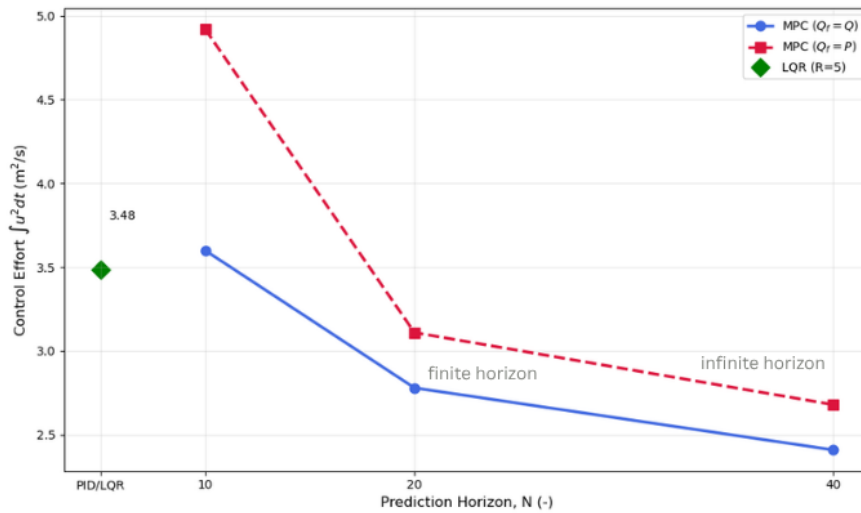


Figure 7.9: Control effort $\int u^2 dt$ versus prediction horizon N for MPC with $Q_f = Q$ and $Q_f = P$, with the LQR ($R = 5$) effort shown as a single reference point.

8 Extension to Moving-Platform Landing

The moving platform is the scenario that motivates the whole study. This chapter formulates the moving-reference tracking problem, presents the analytical argument that distinguishes the controllers on it, and states clearly what has been demonstrated versus what is formulated for the next stage. The empirical controller comparison of Chapter 7 is for the static marker; the moving-platform contribution here is the formulation and the analysis, with any empirical result scoped honestly.

8.1 Scenario and Reference-Tracking Formulation

When the platform translates at (approximately) constant velocity, the landing target is no longer a fixed point but a moving reference. The relative-position error becomes a tracking error against a time-varying setpoint, and the controller must track that setpoint rather than merely regulate a fixed one. Crucially, the platform’s motion does not change the UAV plant: the matrices A_d and B_d of Chapter 4 are unchanged, and the marker velocity enters only through the reference. For MPC this is the forward projection of Eq. (6.5); the same idea underlies the analysis below.

8.2 LQR Steady-State Lag Against a Ramp

The central analytical result is that a plain LQR incurs a constant steady-state lag when tracking a constant-velocity platform. The argument is short and physical. To keep pace with a platform moving at velocity v_m , the UAV must itself hold a steady velocity v_m . In the guidance model this requires a steady, nonzero velocity command; but the LQR command is the static feedback $\mathbf{u} = -K\mathbf{x}$, so a nonzero steady command requires a nonzero steady state — that is, a nonzero steady error. The error therefore cannot converge to zero; it settles to a constant offset (the lag), whose magnitude scales with the platform velocity and inversely with the loop gain.

Equivalently, in classical terms, the closed loop is a type-1 system: its single free integrator (the position state) gives zero steady-state error to a step reference (the static marker) but a constant, finite error to a ramp reference (the constant-velocity marker). This is a structural property of fixed linear state feedback without additional action, not a tuning deficiency; increasing the gain reduces the lag but does not remove it. A feedforward of the marker velocity, or equivalently an integral/servo augmentation, would cancel most of this lag — and, as the next section notes, MPC’s moving reference achieves the anticipatory effect directly.

8.3 MPC Anticipation via the Moving Reference

MPC does not suffer the same structural lag, because it does not rely on a static feedback of the present error alone. By embedding the platform’s projected motion in the reference trajectory over the horizon (Eq. (6.5)), the optimizer plans a sequence of inputs that tracks where the platform *will be*, not only where it is now. The prediction thus acts as an anticipatory feedforward derived from the model and the reference, closing the lag that the reactive law leaves open. This is the second of the two capabilities (with constraint handling) that distinguish MPC from LQR, and it is the reason model-based prediction is the natural choice for the moving-platform case — consistent with the moving-platform literature reviewed in Chapter 2 [19, 12].

8.4 Scope of Results and Discussion

It is stated plainly that the moving-platform contribution of this report is the formulation of Section 8.1 and the analysis of Sections 8.2–8.3, not a completed empirical comparison. The static-marker study establishes the shared linear foundation, the weighting, and the estimator on which a moving-platform study is built; extending the runs to a moving marker — demonstrating the LQR lag empirically and the MPC anticipation against it — is the immediate next step. Framed this way, the chapter contributes a defensible formulation and analysis rather than an overstated result, and it defines exactly the experiment that would complete the comparison.

9 Hardware Commissioning

Hardware commissioning was carried out in parallel with the simulation study on a Holybro S500 quadrotor with a Pixhawk flight controller and an NVIDIA Jetson Orin Nano companion computer. Closed-loop vision-guided landing was not reached within the internship window; however, the full sensor, radio, vision, communication, and state-estimation stack was brought up and validated, and offboard-mode entry was diagnosed to the point of repeatable engagement. The subsections below document what was commissioned and, more usefully, the integration findings and root causes uncovered along the way. These findings define a concrete path to closed-loop flight and are reported as engineering outcomes in their own right.

9.1 Sensor, Motor, and Radio Calibration

Initial bring-up was performed through QGroundControl (QGC). Inertial and magnetic sensors (accelerometer, gyroscope, level-horizon, and compass) were calibrated first, followed by motor calibration covering throttle range and verification of correct motor spin direction and mixer allocation. Radio calibration then mapped transmitter channels to flight functions, with deliberate assignment of flight modes and of the arming and emergency-kill switches. This stage established a reliable “Ready To Fly” baseline (Fig. 9.1).

Flight modes were assigned across the transmitter switches, including a dedicated Offboard mode channel and an emergency-kill channel, so that manual recovery was always available during autonomy tests (Fig. 9.2).

9.2 Vision Pipeline: Exposure Robustness

The most significant vision-side finding concerned camera exposure. Under the factory auto-exposure setting, outdoor operation in direct sunlight drove the sensor to overexposure: the frame washed out to near-white and ArUco marker detection failed entirely (Fig. 9.3, left). Because auto-exposure continuously overrode manual exposure settings, the exposure time could not be constrained until the control mode itself was changed.

The camera was therefore reconfigured to manual exposure at the hardware-minimum exposure time, with reduced brightness and gamma and auto white-balance disabled (Table 9.1). These controls were baked into the camera launch configuration. After tuning, marker detection was stable across a range of outdoor brightness levels, recovering the metric relative pose (x_{rel} , y_{rel} , z_{rel}) required by the landing pipeline (Fig. 9.3, right). Robust exposure values were selected empirically from repeated runs under varying sun conditions.

A carry-over limitation was also documented: the fixed-focus lens exposes no focus or zoom control and loses sharpness above roughly 1 m. This is a physical constraint to be addressed by re-setting the lens focus distance to the operating altitude, and is noted here as an input to future hardware work.

9.3 State Estimation: GPS and Barometer

Two state-estimation issues were diagnosed and resolved. First, GPS availability was intermittent and, even when locked, the EKF2 local-position estimate was unreliable. This was traced to magnetic interference near the receiver; relocating the GPS module toward the rear of the airframe, away from the Jetson Orin Nano, restored a consistent position estimate. Second, the barometer reported altitudes of 9 m to 10 m while the vehicle was physically at 2 m to 3 m; sensor



(a) QGC main window (Ready To Fly).



(b) Radio / RC calibration.

Figure 9.1: Vehicle bring-up in QGroundControl. Sensor, motor, and radio calibration established a stable armed baseline before autonomy testing.



Figure 9.2: Flight-mode and switch assignment, including a dedicated Offboard channel and an emergency-kill switch for safe manual override.

re-calibration removed the offset and the fault did not recur. Both findings are relevant beyond hardware: the false barometer altitude later interacted with offboard-mode logic (Sec. 9.5).

9.4 Companion-Computer Integration and Communication

Offboard control requires a reliable ROS2 link between the Jetson and the flight controller over the micro-XRCE-DDS bridge. The three Pixhawk telemetry ports were allocated as follows: TELEM1 to the telemetry radio, TELEM3 to the RC link, and TELEM2 to the Jetson for the micro-XRCE serial (TX/RX) connection. Establishing this wiring and confirming a stable client-agent session took non-trivial effort.

A recurring pitfall was the `ROS_DOMAIN_ID`: the micro-XRCE client and agent must share the same domain (0 here), and inadvertent changes during remote-desktop sessions broke the link until the domain was standardized back to 0 (Fig. 9.4). In-flight monitoring was performed headlessly over SSH/VNC, since an HDMI display cannot be attached during flight; VNC over a mobile hotspot proved more usable than campus Wi-Fi, though residual latency remains an accepted operational constraint.

9.5 Offboard-Mode Commissioning

Offboard-mode entry was the principal integration hurdle and consumed the largest share of the hardware schedule. Although the controller streamed offboard setpoint ticks at the required rate,

Table 9.1: Tuned camera controls (baked into the camera launch file) versus factory defaults, with the rationale for each change.

Control	Final	Default	Reason for change
<code>auto_exposure</code>	1 (manual)	3 (auto)	Auto-exposure whitewashed the frame; manual mode is also required before the exposure time becomes writable.
<code>exposure_time_absolute</code>	1 (hw min)	313	Shortest exposure — the primary lever against overexposure.
<code>brightness</code>	-64	0	Pulls the overall level down.
<code>gamma</code>	20	67	Darkens mid/highlights to fight the wash-out.
<code>white_balance_automatic</code>	False	True	Auto WB drifted the image toward white after a few frames.

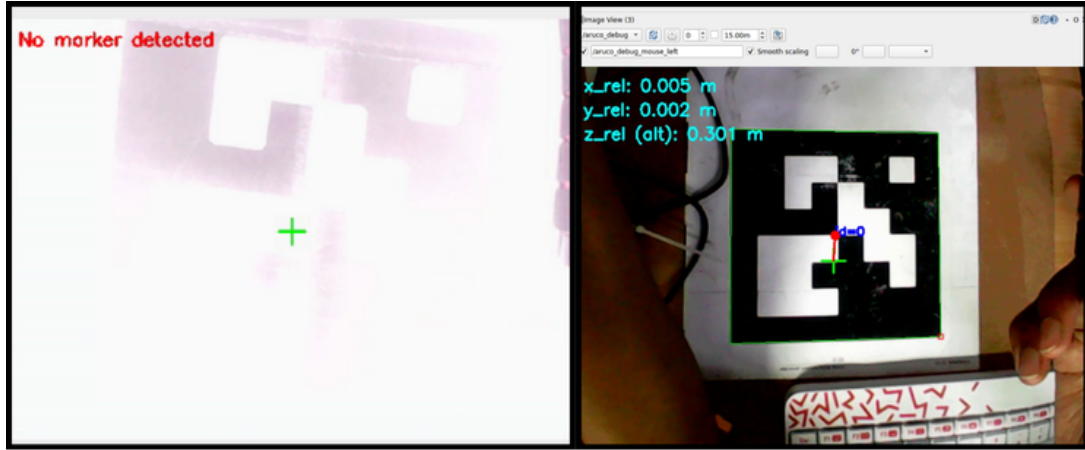


Figure 9.3: Effect of exposure tuning on marker detection. Left: under auto-exposure the frame overexposes and no marker is detected. Right: with manual exposure tuning the marker is reliably detected and its relative pose recovered.

mode engagement was blocked by two coupled faults: blank or incompletely populated trajectory setpoints, and the false barometer altitude of Sec. 9.3, which caused the target setpoint to register as already reached. Stable GPS was an additional precondition. A series of focused test scripts was written to isolate offboard-switching behaviour independently of the full landing pipeline. A secondary issue was centre-of-mass shift on battery replacement: with no dedicated battery mount yet, each swap required a CoM re-check.

9.6 Status and Path to Closed-Loop Trials

At the close of the internship the hardware stack had reached the following state:

- Sensor, motor, and radio calibration complete; stable armed baseline. ✓
- Robust outdoor ArUco detection with metric pose recovery. ✓
- Telemetry-port allocation and micro-XRCE client-agent link established. ✓
- GPS/EKF2 position estimate and barometer faults diagnosed and resolved. ✓
- Offboard-mode entry diagnosed to repeatable engagement; closed-loop vision landing not yet flown.

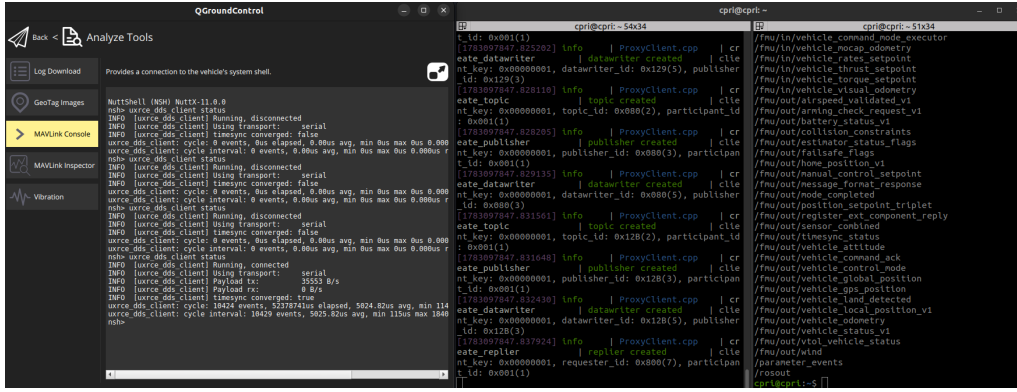


Figure 9.4: Micro XRCE-DDS communication pipeline. The left window shows the PX4 shell confirming the Micro XRCE-DDS client connection with the matching `ROS_DOMAIN_ID`; the middle window shows the running Micro XRCE-DDS Agent; and the right window displays the corresponding PX4 uORB topics successfully bridged and communicated as ROS 2 topics.

The remaining path to a closed-loop trial is well defined: achieve a reliable offboard hold with fully populated setpoints, validate open-loop velocity setpoint tracking against the simulation-tuned controllers, and then close the loop with the vision pipeline for autonomous landing. The simulation study of the preceding chapters provides the controller designs and gains for exactly this step.

10 Discussion

This chapter steps back from the individual results to weigh the controllers against one another, to state the limitations and confounds candidly, and to revisit the modelling assumptions in light of what was observed.

10.1 Controller Trade-offs

The three controllers occupy a clear ordering in capability, but not a simple ordering in desirability; each buys its advantage at a cost. PID is the cheapest to implement and run and needs no model, but it cannot enforce constraints, has no anticipation, and requires manual tuning; it is adequate for the static marker and structurally limited for the moving one. LQR/LQG improves on PID with an optimal, smoothly tuned response for negligible online cost, and its weighting exposes a single interpretable accuracy–effort knob (R); its limitations are that it cannot enforce hard constraints (the cap is imposed only by clipping) and that, as a fixed linear feedback, it lags a moving reference. MPC removes exactly those two limitations — hard constraint enforcement and reference preview — at the cost of solving an optimization every step. The horizon and terminal-cost studies show that this cost is controllable: with the DARE terminal cost, a short horizon performs comparably to a long one, keeping the per-step solve within the real-time budget. The practical reading is that MPC’s advantages matter most precisely where the other two struggle — against constraints and against motion — and are less decisive on the unconstrained static regulation task, where all three land within tolerance.

10.2 Limitations and Confounds

Several limitations bound the strength of the claims. First, and most important for the comparison, the MPC error metrics are computed from the Kalman-estimated state whereas the matched comparison protocol uses ground truth; this difference in error *source* is flagged wherever it appears (Section 7.1) and means the MPC figures are not strictly interchangeable with the ground-truth-referenced ones. Second, the estimator itself is conservative rather than strictly consistent (Chapter 5): its sub-target NIS reflects the first-order model standing in for higher-order dynamics, and a higher-fidelity process model would be needed to tighten it. Third, the moving-platform conclusion rests on analysis and formulation, not on a completed empirical comparison, and should be read as such. Fourth, all results are in simulation; the hardware chapter documents commissioning rather than closed-loop flight, so sim-to-real effects — latency, wind, unmodeled aerodynamics — are not yet measured. Stating these plainly is intended to mark exactly how far the evidence reaches.

10.3 Assumptions Revisited

The plant model of Chapter 4 rests on hover linearization, axis decoupling, and a first-order inner-loop abstraction. The recorded roll and pitch stayed small throughout the landing manoeuvres, so the small-angle linearization is well supported for this regime; the decoupling assumption is likewise reasonable given the yaw was regulated and the axes behaved symmetrically. The first-order lag is the assumption that shows its seams: it is an approximation of PX4’s true higher-order response, and its imperfect fit surfaces both in the identified time constant (which is inflated when the autopilot’s rate limiters are active) and in the conservative estimator consistency. These are honest limitations of a deliberately minimal, control-oriented model — the same minimality

that makes the model linear, low-order, and identical across all three controllers, and hence makes the fair comparison possible. Improving model fidelity is therefore the assumption most worth revisiting, and it connects directly to the future work that follows.

11 Conclusions and Future Work

11.1 Conclusions

This work presented the development of a vision-guided precision landing framework for a multirotor UAV using the PX4 autopilot and ROS 2 ecosystem. A complete architecture integrating onboard vision, offboard control, and companion computing was established, enabling autonomous precision landing using ArUco-based relative pose estimation.

A control-oriented UAV model was developed to implement and compare PID, Linear Quadratic Regulator (LQR), and Model Predictive Control (MPC) under a common framework. A Kalman filter was designed for state estimation from vision measurements and validated using the Normalized Innovation Squared (NIS) criterion. Simulation studies demonstrated that all controllers successfully achieved precision landing on a stationary target, while highlighting the advantages of optimal and predictive control in terms of response quality and constraint handling.

In addition to simulation, the hardware platform was successfully commissioned through sensor calibration, PX4–ROS 2 communication, Micro XRCE-DDS integration, and vision system development. Although fully autonomous hardware landing was beyond the scope of the internship, the developed framework provides a robust foundation for future experimental validation and deployment.

11.2 Future Work

Future work will focus on extending the proposed framework to autonomous landing on a moving UGV platform through experimental validation. The moving-platform formulation presented in this report can be integrated directly with the existing MPC framework to evaluate tracking performance under dynamic landing scenarios.

Further work includes completing closed-loop hardware flight testing, incorporating higher-fidelity vehicle dynamics and disturbance models, and enhancing the vision subsystem through sensor fusion with depth cameras or visual-inertial odometry. The developed architecture also provides a basis for future research on cooperative UAV–UGV systems for autonomous inspection, logistics, and related field applications.

Bibliography

- [1] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, 2012.
- [2] B. Friedland, *Control System Design: An Introduction to State-Space Methods*. McGraw-Hill, 1986.
- [3] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.
- [4] R. Mahony, V. Kumar, and P. Corke, “Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 3, pp. 20–32, Sept. 2012.
- [5] H. Cengiz and M. Uzun, “A review of quadrotor unmanned aerial vehicle control strategies,” *Simulation Modelling Practice and Theory*, vol. 118, 2022.
- [6] PX4 Development Team, “PX4 Autopilot Documentation.” Available: <https://docs.px4.io>
- [7] Open Robotics, “ROS 2 Documentation.” Available: <https://docs.ros.org/en/humble>
- [8] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi: A software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [9] Fonyuy45, “Quadrotor Modelling, Control and Estimation,” GitHub repository. Available: https://github.com/Fonyuy45/quadrotor_modelling_control_estimation
- [10] V. Palacherla, “Quadrotor Control with LQR and iLQR,” GitHub repository. Available: <https://github.com/venkataPalacherla/Quadrotor-Control-with-LQR-iLQR>
- [11] Y. Guo *et al.*, “Relative Precision Point Position Control for Quadrotor UAV Landing,” *Journal of System Simulation*, vol. 34, no. 11, 2022.
- [12] “Autonomous Landing of a Quadrotor on a Moving Platform Using a Motion Capture System.” Available: <https://www.bohrium.com/en/paper-details/autonomous-landing-of-a-quadrotor-on-a-moving-platform-using-motion-capture-system/1004563040244334594-60748>
- [13] “Model Predictive Control for Autonomous Quadrotor Landing,” arXiv:2505.03931, 2025.
- [14] “Model Predictive Control of Quadrotor UAV,” Tribhuvan University Thesis. Available: <https://elibrary.tucl.edu.np/items/283d08d8-afc3-491d-8001-2182d12f96b7>
- [15] L. Qing, “MPC Quadcopter,” GitHub repository. Available: <https://github.com/linqingbh/MPC-quadcopter>
- [16] M. Herraz, “MPC Quadrotor,” GitHub repository. Available: https://github.com/MahfoudHerraz/mpc_quadrotor
- [17] MathWorks, “Control of Quadrotor Using Nonlinear Model Predictive Control,” MPC Toolbox Documentation. Available: <https://www.mathworks.com/help/mpc/ug/control-of-quadrotor-using-nonlinear-model-predictive-control.html>

- [18] “Nonlinear Model Predictive Control for Quadrotor UAVs,” *IET Cyber-Systems and Robotics*, 2021. doi:10.1049/ccs2.12081.
- [19] “Precision Landing of a Quadrotor on a Moving Platform Using Robust Control,” *Control Engineering Practice*, 2022.
- [20] “Vision-Based Precision Landing of a Quadrotor Using PID Control,” arXiv:2209.14436, 2022.
- [21] 80L Robotics, “Precision Landing,” GitHub repository. Available: <https://github.com/80L-Robotics/precision-landing>